

CHƯƠNG 16. XỬ LÝ NGÔN NGỮ TỰ NHIÊN VỚI RNN VÀ CƠ CHẾ CHÚ Ý

Khi Alan Turing hình dung bài kiểm tra Turing nổi tiếng của mình vào năm 1950, ông đã đề xuất một cách để đánh giá khả năng của một cỗ máy trong việc sánh ngang với trí thông minh của con người. Ông có thể đã kiểm tra nhiều thứ, chẳng hạn như khả năng nhận diện mèo trong hình ảnh, chơi cờ vua, sáng tác nhạc hoặc thoát khỏi mê cung, nhưng điều thú vị là ông đã chọn một nhiệm vụ ngôn ngữ.

Cụ thể hơn, ông đã nghĩ ra một chatbot có khả năng đánh lừa người đối thoại nghĩ rằng đó là con người. Bài kiểm tra này có những điểm yếu của nó: một tập hợp các quy tắc được mã hóa cứng có thể đánh lừa những người cả tin hoặc ngây thơ (ví dụ: cỗ máy có thể đưa ra các câu trả lời chung chung đã được định nghĩa trước để phản hồi một số từ khóa, nó có thể giả vờ đang nói đùa hoặc say xin để vượt qua những câu trả lời kỳ lạ nhất của nó, hoặc nó có thể tránh những câu hỏi khó bằng cách trả lời chúng bằng chính câu hỏi của mình), và nhiều khía cạnh của trí thông minh con người hoàn toàn bị bỏ qua (ví dụ: khả năng diễn giải giao tiếp phi ngôn ngữ như biểu cảm khuôn mặt, hoặc học một công việc thủ công). Nhưng bài kiểm tra này đã làm nổi bật một thực tế rằng việc nắm vững ngôn ngữ được cho là khả năng nhận thức vĩ đại nhất của loài Homo sapiens.

Liệu chúng ta có thể xây dựng một cỗ máy có thể nắm vững ngôn ngữ viết và nói? Đây là mục tiêu cuối cùng của nghiên cứu NLP, nhưng nó hơi quá rộng, vì vậy trên thực tế, các nhà nghiên cứu tập trung vào các nhiệm vụ cụ thể hơn, chẳng hạn như phân loại văn bản, dịch thuật, tóm tắt, trả lời câu hỏi và nhiều hơn nữa. Một cách tiếp cận phổ biến cho các nhiệm vụ ngôn ngữ tự nhiên là sử dụng mạng nơ-ron hồi quy. Do đó, chúng ta sẽ tiếp tục khám phá RNN (đã được giới thiệu trong Chương 15), bắt đầu với một RNN ký tự, hay char-RNN, được huấn luyện để dự đoán ký tự tiếp theo trong một câu. Điều này sẽ cho phép chúng ta tạo ra một số văn bản gốc. Chúng ta sẽ sử dụng một RNN không trạng thái (học trên các phần văn bản ngẫu nhiên ở mỗi lần lặp, không có bất kỳ thông tin nào về phần còn lại của văn bản), sau đó chúng ta sẽ xây dựng một RNN có trạng thái (giữ lại trạng thái ẩn giữa các lần lặp huấn luyện và tiếp tục đọc từ nơi nó dừng lại, cho phép nó học các mẫu dài hơn). Tiếp theo, chúng ta sẽ xây dựng một RNN để thực hiện phân tích cảm xúc (ví dụ: đọc các bài đánh giá phim và trích xuất cảm nhận của người đánh giá về bộ phim), lần này xử lý các câu như chuỗi các từ, thay vì các ký tự. Sau đó, chúng ta sẽ trình bày cách RNN có thể được sử dụng để xây dựng kiến trúc mã hóa-giải mã có khả năng thực hiện dịch máy thần kinh (NMT), dịch tiếng Anh sang tiếng Tây Ban Nha.

Trong phần thứ hai của chương này, chúng ta sẽ khám phá các cơ chế chú ý. Đúng như tên gọi của chúng, đây là các thành phần mạng nơ-ron học cách chọn phần đầu vào mà phần còn lại của mô hình nên tập trung vào ở mỗi bước thời gian. Đầu tiên, chúng ta sẽ tăng hiệu suất của kiến trúc mã hóa-giải mã dựa trên RNN bằng cách sử dụng cơ chế chú ý. Tiếp theo, chúng ta sẽ bỏ qua RNN hoàn toàn và sử dụng một kiến trúc chỉ có cơ chế chú ý rất thành công, được gọi là transformer, để xây dựng một mô hình dịch thuật. Sau đó, chúng ta sẽ thảo luận về một số tiến bộ quan trọng nhất trong NLP trong vài năm gần đây, bao gồm các mô

hình ngôn ngữ cực kỳ mạnh mẽ như GPT và BERT, cả hai đều dựa trên transformer. Cuối cùng, tôi sẽ chỉ cho bạn cách bắt đầu với thư viện Transformers xuất sắc của Hugging Face.

Hãy bắt đầu với một mô hình đơn giản và thú vị có thể viết giống như Shakespeare (đại loại thế).

Tạo văn bản kiểu Shakespeare bằng cách sử dụng RNN ký tự

Trong một bài đăng blog nổi tiếng năm 2015 có tựa đề “Hiệu quả bất hợp lý của mạng nơ-ron hồi quy”, Andrej Karpathy đã chỉ ra cách huấn luyện một RNN để dự đoán ký tự tiếp theo trong một câu. Char-RNN này sau đó có thể được sử dụng để tạo văn bản mới, từng ký tự một. Đây là một mẫu nhỏ văn bản được tạo bởi một mô hình char-RNN sau khi nó được huấn luyện trên tất cả các tác phẩm của Shakespeare:

PANDARUS: Ôi, tôi nghĩ anh ta sẽ đến gần và ngày đó Khi cơn mưa nhỏ sẽ được ban cho để không bao giờ được nuôi dưỡng, Và ai là một chuỗi và đối tượng cái chết của anh ta, Tôi sẽ không ngủ.

Không hẳn là một kiệt tác, nhưng điều ấn tượng là mô hình đã có thể học được từ ngữ, ngữ pháp, dấu câu đúng và hơn thế nữa, chỉ bằng cách học cách dự đoán ký tự tiếp theo trong một câu. Đây là ví dụ đầu tiên của chúng ta về một mô hình ngôn ngữ; các mô hình ngôn ngữ tương tự (nhưng mạnh mẽ hơn nhiều), được thảo luận sau trong chương này, là cốt lõi của NLP hiện đại. Trong phần còn lại của phần này, chúng ta sẽ xây dựng một char-RNN từng bước, bắt đầu bằng việc tạo tập dữ liệu.

Tạo tập dữ liệu huấn luyện

Đầu tiên, sử dụng hàm tiện ích `tf.keras.utils.get_file()` của Keras, hãy tải xuống tất cả các tác phẩm của Shakespeare. Dữ liệu được tải từ dự án char-rnn của Andrej Karpathy:

```
import tensorflow as tf

shakespeare_url = "https://hml.info/shakespeare" # URL viết tắt
filepath = tf.keras.utils.get_file("shakespeare.txt", shakespeare_url)
with open(filepath) as f:
    shakespeare_text = f.read()
```

Hãy in vài dòng đầu tiên:

```
>>> print(shakespeare_text[:80])
First Citizen:
Before we proceed any further, hear me speak.

All:
Speak, speak.
```

Trông giống Shakespeare rồi!

Tiếp theo, chúng ta sẽ sử dụng một lớp `tf.keras.layers.TextVectorization` (đã được giới thiệu trong Chương 13) để mã hóa văn bản này. Chúng ta đặt `split="character"` để có mã hóa cấp ký tự thay vì mã hóa cấp từ mặc định, và chúng ta sử dụng `standardize="lower"` để chuyển đổi văn bản sang chữ thường (điều này sẽ đơn giản hóa nhiệm vụ):

```
text_vec_layer = tf.keras.layers.TextVectorization(split="character",
                                                    standardize="lower")
text_vec_layer.adapt([shakespeare_text])
encoded = text_vec_layer([shakespeare_text])[0]
```

Mỗi ký tự hiện được ánh xạ tới một số nguyên, bắt đầu từ 2. Lớp `TextVectorization` đã dành giá trị 0 cho các token đệm, và nó dành 1 cho các ký tự không xác định. Hiện tại chúng ta sẽ không cần cả hai token này, vì vậy hãy trừ 2 từ các ID ký tự và tính toán số lượng ký tự riêng biệt và tổng số ký tự:

```
encoded -= 2 # bỏ các token 0 (đệm) và 1 (không xác định), mà chúng ta sẽ không sử dụng
n_tokens = text_vec_layer.vocabulary_size() - 2 # số ký tự riêng biệt = 39
dataset_size = len(encoded) # tổng số ký tự = 1,115,394
```

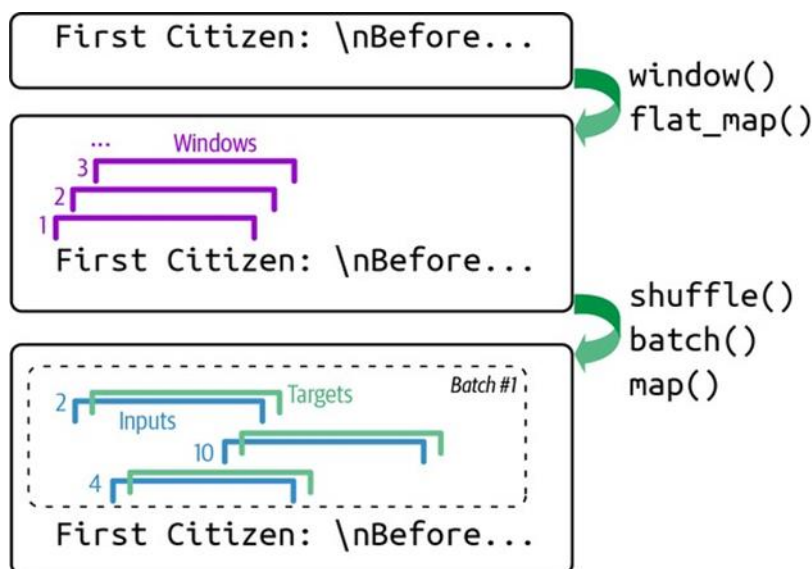
Tiếp theo, giống như chúng ta đã làm trong Chương 15, chúng ta có thể biến chuỗi rất dài này thành một tập dữ liệu các cửa sổ mà sau đó chúng ta có thể sử dụng để huấn luyện một RNN tuần tự. Các mục tiêu sẽ tương tự như đầu vào, nhưng được dịch chuyển một bước thời gian sang “tương lai”. Ví dụ, một mẫu trong tập dữ liệu có thể là một chuỗi các ID ký tự đại diện cho văn bản “to be or not to b” (không có chữ “e” cuối cùng), và mục tiêu tương ứng — một chuỗi các ID ký tự đại diện cho văn bản “o be or not to be” (với chữ “e” cuối cùng, nhưng không có chữ “t” đầu). Hãy viết một hàm tiện ích nhỏ để chuyển đổi một chuỗi dài các ID ký tự thành một tập dữ liệu các cặp cửa sổ đầu vào/mục tiêu:

```
def to_dataset(sequence, length, shuffle=False, seed=None, batch_size=32):
    ds = tf.data.Dataset.from_tensor_slices(sequence)
    ds = ds.window(length + 1, shift=1, drop_remainder=True)
    ds = ds.flat_map(lambda window_ds: window_ds.batch(length + 1))
    if shuffle:
        ds = ds.shuffle(buffer_size=100_000, seed=seed)
    ds = ds.batch(batch_size)
    return ds.map(lambda window: (window[:, :-1], window[:, 1:])).prefetch(1)
```

Hàm này bắt đầu khá giống với hàm tiện ích tùy chỉnh `to_windows()` mà chúng ta đã tạo trong Chương 15:

- Nó nhận một chuỗi làm đầu vào (tức là văn bản đã được mã hóa), và tạo ra một tập dữ liệu chứa tất cả các cửa sổ có độ dài mong muốn.
- Nó tăng độ dài lên một, vì chúng ta cần ký tự tiếp theo cho mục tiêu.
- Sau đó, nó xáo trộn các cửa sổ (tùy chọn), gộp chúng lại, chia chúng thành các cặp đầu vào/đầu ra, và kích hoạt prefetching.

Hình 16-1 tóm tắt các bước chuẩn bị tập dữ liệu: nó hiển thị các cửa sổ có độ dài 11, và kích thước batch là 3. Chỉ số bắt đầu của mỗi cửa sổ được chỉ ra bên cạnh nó.



Hình 16-1. Chuẩn bị tập dữ liệu các cửa sổ đã xáo trộn

Bây giờ chúng ta đã sẵn sàng tạo tập huấn luyện, tập xác thực và tập kiểm tra. Chúng ta sẽ sử dụng khoảng 90% văn bản để huấn luyện, 5% để xác thực và 5% để kiểm tra:

```
length = 100
tf.random.set_seed(42)

train_set = to_dataset(encoded[:1_000_000], length=length, shuffle=True, seed=42)
valid_set = to_dataset(encoded[1_000_000:1_060_000], length=length)
test_set = to_dataset(encoded[1_060_000:], length=length)
```

Thế là xong! Chuẩn bị tập dữ liệu là phần khó nhất. Bây giờ hãy tạo mô hình.

Xây dựng và huấn luyện mô hình Char-RNN

Vì tập dữ liệu của chúng ta khá lớn, và việc mô hình hóa ngôn ngữ là một nhiệm vụ khá khó khăn, chúng ta cần nhiều hơn một RNN đơn giản với vài nơ-ron hồi quy. Hãy xây dựng và huấn luyện một mô hình với một lớp GRU bao gồm 128 đơn vị (bạn có thể thử điều chỉnh số lượng lớp và đơn vị sau, nếu cần):

```
model = tf.keras.Sequential([
    tf.keras.layers.Embedding(input_dim=n_tokens, output_dim=16),
    tf.keras.layers.GRU(128, return_sequences=True),
    tf.keras.layers.Dense(n_tokens, activation="softmax")
])
model.compile(loss="sparse_categorical_crossentropy",
              optimizer="nadam", metrics=["accuracy"])
model_ckpt = tf.keras.callbacks.ModelCheckpoint(
```

```
"my_shakespeare_model", monitor="val_accuracy", save_best_only=True)
history = model.fit(train_set, validation_data=valid_set, epochs=10,
                    callbacks=[model_ckpt])
```

Hãy xem qua đoạn code này:

- Chúng ta sử dụng một lớp Embedding làm lớp đầu tiên, để mã hóa các ID ký tự (embedding đã được giới thiệu trong Chương 13). Số chiều đầu vào của lớp Embedding là số ID ký tự riêng biệt, và số chiều đầu ra là một siêu tham số bạn có thể điều chỉnh — chúng ta sẽ đặt nó là 16. Trong khi đầu vào của lớp Embedding sẽ là các tensor 2D có hình dạng [batch size, window length], đầu ra của lớp Embedding sẽ là một tensor 3D có hình dạng [batch size, window length, embedding size].
- Chúng ta sử dụng một lớp Dense cho lớp đầu ra: nó phải có 39 đơn vị (n_tokens) vì có 39 ký tự riêng biệt trong văn bản, và chúng ta muốn xuất ra một xác suất cho mỗi ký tự có thể có (ở mỗi bước thời gian). 39 xác suất đầu ra nên tổng cộng là 1 ở mỗi bước thời gian, vì vậy chúng ta áp dụng hàm kích hoạt softmax cho đầu ra của lớp Dense.
- Cuối cùng, chúng ta biên dịch mô hình này, sử dụng hàm mất mát "sparse_categorical_crossentropy" và bộ tối ưu hóa Nadam, và chúng ta huấn luyện mô hình trong vài epoch, sử dụng callback ModelCheckpoint để lưu mô hình tốt nhất (về độ chính xác xác thực) khi quá trình huấn luyện diễn ra.

Mô hình này không xử lý tiền xử lý văn bản, vì vậy hãy bọc nó trong một mô hình cuối cùng chứa lớp `tf.keras.layers.TextVectorization` làm lớp đầu tiên, cộng với một lớp `tf.keras.layers.Lambda` để trừ 2 từ các ID ký tự vì hiện tại chúng ta không sử dụng các token đệm và không xác định:

```
shakespeare_model = tf.keras.Sequential([
    text_vec_layer,
    tf.keras.layers.Lambda(lambda X: X - 2), # không có token <PAD> hoặc <UNK>
>
    model
])
```

Và bây giờ hãy sử dụng nó để dự đoán ký tự tiếp theo trong một câu:

```
>>> y_proba = shakespeare_model.predict(["To be or not to b"])[0, -1]
>>> y_pred = tf.argmax(y_proba) # chọn ID ký tự có khả năng nhất
>>> text_vec_layer.get_vocabulary()[y_pred + 2]
'e'
```

Tuyệt vời, mô hình đã dự đoán đúng ký tự tiếp theo. Bây giờ hãy sử dụng mô hình này để giả vờ chúng ta là Shakespeare!

Tạo văn bản Shakespeare giả

Để tạo văn bản mới bằng cách sử dụng mô hình char-RNN, chúng ta có thể cung cấp cho nó một số văn bản, để mô hình dự đoán ký tự tiếp theo có khả năng nhất, thêm nó vào cuối văn bản, sau đó cung cấp văn bản mở rộng cho mô hình để đoán ký tự tiếp theo, và cứ thế. Điều

này được gọi là giải mã tham lam (greedy decoding). Nhưng trong thực tế, điều này thường dẫn đến việc các từ giống nhau bị lặp đi lặp lại. Thay vào đó, chúng ta có thể lấy mẫu ký tự tiếp theo một cách ngẫu nhiên, với xác suất bằng xác suất ước tính, bằng cách sử dụng hàm `tf.random.categorical()` của TensorFlow. Điều này sẽ tạo ra văn bản đa dạng và thú vị hơn. Hàm `categorical()` lấy mẫu các chỉ số lớp ngẫu nhiên, cho các log xác suất lớp (logits). Ví dụ:

```
>>> log_probabilities = tf.math.log([[0.5, 0.4, 0.1]]) # xác suất = 50%, 40%, và 10%
>>> tf.random.set_seed(42)
>>> tf.random.categorical(log_probabilities, num_samples=8) # lấy 8 mẫu
<tf.Tensor: shape=(1, 8), dtype=int64, numpy=array([[0, 1, 0, 2, 1, 0, 0, 1]])
>>
```

Để kiểm soát tốt hơn sự đa dạng của văn bản được tạo ra, chúng ta có thể chia các logits cho một số được gọi là nhiệt độ (temperature), mà chúng ta có thể điều chỉnh tùy ý. Nhiệt độ gần bằng 0 sẽ ưu tiên các ký tự có xác suất cao, trong khi nhiệt độ cao sẽ cho tất cả các ký tự một xác suất bằng nhau. Nhiệt độ thấp thường được ưu tiên khi tạo văn bản khá cứng nhắc và chính xác, chẳng hạn như các phương trình toán học, trong khi nhiệt độ cao hơn được ưu tiên khi tạo văn bản đa dạng và sáng tạo hơn.

Hàm trợ giúp tùy chỉnh `next_char()` tiếp theo sử dụng phương pháp này để chọn ký tự tiếp theo để thêm vào văn bản đầu vào:

```
def next_char(text, temperature=1):
    y_proba = shakespeare_model.predict([text])[0, -1:]
    rescaled_logits = tf.math.log(y_proba) / temperature
    char_id = tf.random.categorical(rescaled_logits, num_samples=1)[0, 0]
    return text_vec_layer.get_vocabulary()[char_id + 2]
```

Tiếp theo, chúng ta có thể viết một hàm trợ giúp nhỏ khác sẽ liên tục gọi `next_char()` để lấy ký tự tiếp theo và thêm nó vào văn bản đã cho:

```
def extend_text(text, n_chars=50, temperature=1):
    for _ in range(n_chars):
        text += next_char(text, temperature)
    return text
```

Bây giờ chúng ta đã sẵn sàng tạo văn bản! Hãy thử với các giá trị nhiệt độ khác nhau:

```
>>> tf.random.set_seed(42)

>>> print(extend_text("To be or not to be", temperature=0.01))
To be or not to be the duke
as it is a proper
strange death, and the

>>> print(extend_text("To
be or not to be", temperature=1)) To be or
not to behold?
```

```
second push:
gremio, lord all, a sistermen,

>>> print(extend_text("To be or not to be", temperature=100))
To be or not to bef
,mt'&o3fpadm!$ wh!nse?bws3est--vgerdjw?c-y-ewznq
```

Shakespeare dường như đang chịu đựng một đợt nắng nóng. Để tạo văn bản thuyết phục hơn, một kỹ thuật phổ biến là chỉ lấy mẫu từ k ký tự hàng đầu, hoặc chỉ từ tập hợp nhỏ nhất các ký tự hàng đầu có tổng xác suất vượt quá một ngưỡng nhất định (điều này được gọi là lấy mẫu hạt nhân). Ngoài ra, bạn có thể thử sử dụng tìm kiếm chùm (beam search), mà chúng ta sẽ thảo luận sau trong chương này, hoặc sử dụng nhiều lớp GRU hơn và nhiều nơ-ron hơn cho mỗi lớp, huấn luyện lâu hơn và thêm một số chuẩn hóa nếu cần. Cũng lưu ý rằng mô hình hiện không thể học các mẫu dài hơn độ dài `length`, tức là chỉ 100 ký tự. Bạn có thể thử làm cho cửa sổ này lớn hơn, nhưng nó cũng sẽ làm cho việc huấn luyện khó hơn, và ngay cả các ô LSTM và GRU cũng không thể xử lý các chuỗi rất dài. Một cách tiếp cận thay thế là sử dụng một RNN có trạng thái.

RNN có trạng thái

Cho đến nay, chúng ta chỉ sử dụng các RNN không trạng thái: ở mỗi lần lặp huấn luyện, mô hình bắt đầu với một trạng thái ẩn đầy các số 0, sau đó nó cập nhật trạng thái này ở mỗi bước thời gian, và sau bước thời gian cuối cùng, nó loại bỏ trạng thái đó vì không cần nữa. Điều gì sẽ xảy ra nếu chúng ta hướng dẫn RNN giữ lại trạng thái cuối cùng này sau khi xử lý một batch huấn luyện và sử dụng nó làm trạng thái ban đầu cho batch huấn luyện tiếp theo? Bằng cách này, mô hình có thể học các mẫu dài hạn mặc dù chỉ lan truyền ngược qua các chuỗi ngắn. Đây được gọi là RNN có trạng thái. Hãy xem cách xây dựng một mô hình như vậy.

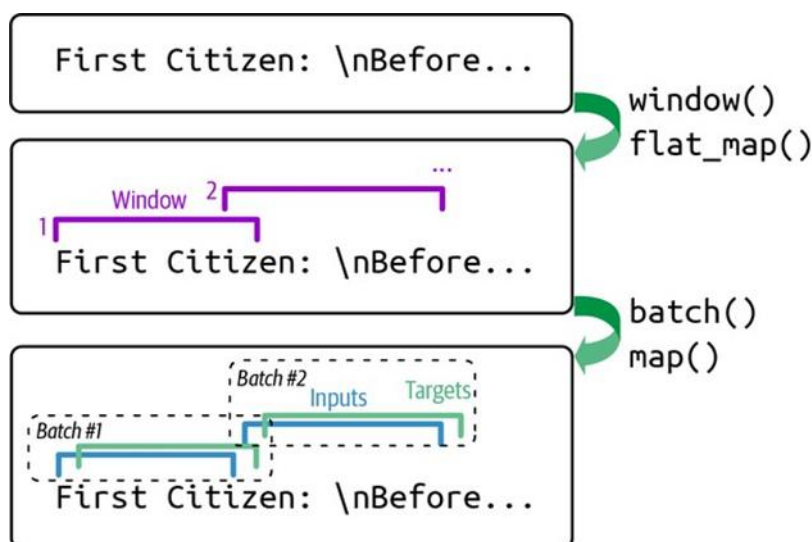
Đầu tiên, lưu ý rằng một RNN có trạng thái chỉ có ý nghĩa nếu mỗi chuỗi đầu vào trong một batch bắt đầu chính xác từ nơi chuỗi tương ứng trong batch trước đó dừng lại. Vì vậy, điều đầu tiên chúng ta cần làm để xây dựng một RNN có trạng thái là sử dụng các chuỗi đầu vào tuần tự và không chồng chéo (thay vì các chuỗi bị xáo trộn và chồng chéo mà chúng ta đã sử dụng để huấn luyện các RNN không trạng thái). Khi tạo `tf.data.Dataset`, chúng ta phải sử dụng `shift=length` (thay vì `shift=1`) khi gọi phương thức `window()`. Hơn nữa, chúng ta không được gọi phương thức `shuffle()`.

Rất tiếc, việc gộp batch khó hơn nhiều khi chuẩn bị tập dữ liệu cho một RNN có trạng thái so với RNN không trạng thái. Thật vậy, nếu chúng ta gọi `batch(32)`, thì 32 cửa sổ liên tiếp sẽ được đặt trong cùng một batch, và batch tiếp theo sẽ không tiếp tục mỗi cửa sổ này từ nơi nó dừng lại. Batch đầu tiên sẽ chứa các cửa sổ từ 1 đến 32 và batch thứ hai sẽ chứa các cửa sổ từ 33 đến 64, vì vậy nếu bạn xem xét, ví dụ, cửa sổ đầu tiên của mỗi batch (tức là cửa sổ 1 và 33), bạn có thể thấy rằng chúng không liên tiếp. Giải pháp đơn giản nhất cho vấn đề này là chỉ sử dụng kích thước batch là 1. Hàm tiện ích tùy chỉnh `to_dataset_for_stateful_rnn()` sau đây sử dụng chiến lược này để chuẩn bị tập dữ liệu cho một RNN có trạng thái:

```
def to_dataset_for_stateful_rnn(sequence, length):
    ds = tf.data.Dataset.from_tensor_slices(sequence)
    ds = ds.window(length + 1, shift=length, drop_remainder=True)
    ds = ds.flat_map(lambda window: window.batch(length + 1)).batch(1)
    return ds.map(lambda window: (window[:, :-1], window[:, 1:])).prefetch(1)

stateful_train_set = to_dataset_for_stateful_rnn(encoded[:1_000_000], length)
stateful_valid_set = to_dataset_for_stateful_rnn(encoded[1_000_000:1_060_000]
,
length)
stateful_test_set = to_dataset_for_stateful_rnn(encoded[1_060_000:], length)
```

Hình 16-2 tóm tắt các bước chính của hàm này.



Hình 16-2. Chuẩn bị tập dữ liệu các đoạn chuỗi liên tiếp cho một RNN có trạng thái

Việc gộp batch khó hơn, nhưng không phải là không thể. Ví dụ, chúng ta có thể chia văn bản của Shakespeare thành 32 văn bản có độ dài bằng nhau, tạo một tập dữ liệu các chuỗi đầu vào liên tiếp cho mỗi văn bản, và cuối cùng sử dụng `tf.data.Dataset.zip(datasets).map(lambda *windows: tf.stack(windows))` để tạo các batch liên tiếp phù hợp, trong đó chuỗi đầu vào thứ *n* trong một batch bắt đầu chính xác từ nơi chuỗi đầu vào thứ *n* kết thúc trong batch trước đó (xem sổ tay để biết toàn bộ mã).

Bây giờ, hãy tạo RNN có trạng thái. Chúng ta cần đặt đối số `stateful` thành `True` khi tạo mỗi lớp hồi quy, và vì RNN có trạng thái cần biết kích thước batch (vì nó sẽ giữ một trạng thái cho mỗi chuỗi đầu vào trong batch). Do đó, chúng ta phải đặt đối số `batch_input_shape` trong lớp đầu tiên. Lưu ý rằng chúng ta có thể để chiều thứ hai không xác định, vì các chuỗi đầu vào có thể có bất kỳ độ dài nào:

```
model = tf.keras.Sequential([
    tf.keras.layers.Embedding(input_dim=n_tokens, output_dim=16,
                              batch_input_shape=[1, None]),
    tf.keras.layers.GRU(128, return_sequences=True, stateful=True),
```



```
tf.keras.layers.Dense(n_tokens, activation="softmax")
])
```

Vào cuối mỗi epoch, chúng ta cần đặt lại các trạng thái trước khi quay lại đầu văn bản. Để làm điều này, chúng ta có thể sử dụng một callback Keras tùy chỉnh nhỏ:

```
class ResetStatesCallback(tf.keras.callbacks.Callback):
    def on_epoch_begin(self, epoch, logs):
        self.model.reset_states()
```

Và bây giờ chúng ta có thể biên dịch mô hình và huấn luyện nó bằng cách sử dụng callback của chúng ta:

```
model.compile(loss="sparse_categorical_crossentropy",
              optimizer="nadam", metrics=["accuracy"])
history = model.fit(stateful_train_set, validation_data=stateful_valid_set,
                  epochs=10, callbacks=[ResetStatesCallback(), model_checkpoint])
```

Điều thú vị là, mặc dù mô hình char-RNN chỉ được huấn luyện để dự đoán ký tự tiếp theo, nhiệm vụ tưởng chừng đơn giản này thực sự yêu cầu nó học một số nhiệm vụ cấp cao hơn. Ví dụ, để tìm ký tự tiếp theo sau “Great movie, I really”, sẽ rất hữu ích nếu hiểu rằng câu đó là tích cực, vì vậy điều tiếp theo có nhiều khả năng là chữ “l” (cho “loved”) hơn là “h” (cho “hated”). Trên thực tế, một bài báo năm 2017 của Alec Radford và các nhà nghiên cứu OpenAI khác mô tả cách các tác giả đã huấn luyện một mô hình giống char-RNN lớn trên một tập dữ liệu lớn, và thấy rằng một trong các nơ-ron hoạt động như một bộ phân loại phân tích cảm xúc xuất sắc: mặc dù mô hình được huấn luyện mà không có bất kỳ nhãn nào, nơ-ron cảm xúc — như họ gọi nó — đã đạt được hiệu suất tiên tiến trên các tiêu chuẩn phân tích cảm xúc. Điều này đã báo trước và thúc đẩy việc tiền huấn luyện không giám sát trong NLP.

Nhưng trước khi chúng ta khám phá tiền huấn luyện không giám sát, hãy chuyển sự chú ý của chúng ta sang các mô hình cấp từ và cách sử dụng chúng theo cách giám sát để phân tích cảm xúc. Trong quá trình này, bạn sẽ học cách xử lý các chuỗi có độ dài thay đổi bằng cách sử dụng mặt nạ (masking).

Phân tích cảm xúc

Tạo văn bản có thể thú vị và mang tính hướng dẫn, nhưng trong các dự án thực tế, một trong những ứng dụng phổ biến nhất của NLP là phân loại văn bản — đặc biệt là phân tích cảm xúc. Nếu phân loại hình ảnh trên tập dữ liệu MNIST là “Hello world!” của thị giác máy tính, thì phân tích cảm xúc trên tập dữ liệu đánh giá IMDb là “Hello world!” của xử lý ngôn ngữ tự nhiên. Tập dữ liệu IMDb bao gồm 50.000 đánh giá phim bằng tiếng Anh (25.000 để huấn luyện, 25.000 để kiểm tra) được trích xuất từ Internet Movie Database nổi tiếng, cùng với một mục tiêu nhị phân đơn giản cho mỗi đánh giá cho biết nó là tiêu cực (0) hay tích cực (1). Giống như MNIST, tập dữ liệu đánh giá IMDb phổ biến vì những lý do chính đáng: nó đủ đơn giản để được giải quyết trên một máy tính xách tay trong một khoảng thời gian hợp lý, nhưng đủ thách thức để trở nên thú vị và bổ ích.

Hãy tải tập dữ liệu IMDb bằng thư viện TensorFlow Datasets (đã được giới thiệu trong Chương 13). Chúng ta sẽ sử dụng 90% đầu tiên của tập huấn luyện để huấn luyện, và 10% còn lại để xác thực:

```
import tensorflow_datasets as tfds

raw_train_set, raw_valid_set, raw_test_set = tfds.load(
    name="imdb_reviews",
    split=["train[:90%]", "train[90%:]", "test"],
    as_supervised=True
)
tf.random.set_seed(42)
train_set = raw_train_set.shuffle(5000, seed=42).batch(32).prefetch(1)
valid_set = raw_valid_set.batch(32).prefetch(1)
test_set = raw_test_set.batch(32).prefetch(1)
```

Hãy kiểm tra một vài đánh giá:

```
>>> for review, label in raw_train_set.take(4):
...     print(review.numpy().decode("utf-8"))
...     print("Label:", label.numpy())
...
This was an absolutely
terrible movie. Don't be lured in by Christopher [...]
Label: 0
I have been known to
fall asleep during films, but this is usually due to [...]
Label: 0
Mann photographs the
Alberta Rocky Mountains in a superb fashion, and [...]
Label: 0
This is the kind of
film for a snowy Sunday afternoon when the rest of the [...]
Label: 1
```

Một số đánh giá dễ phân loại. Ví dụ, đánh giá đầu tiên bao gồm các từ “terrible movie” ngay trong câu đầu tiên. Nhưng trong nhiều trường hợp, mọi thứ không đơn giản như vậy. Ví dụ, đánh giá thứ ba bắt đầu một cách tích cực, mặc dù cuối cùng nó là một đánh giá tiêu cực (nhãn 0).

Để xây dựng một mô hình cho nhiệm vụ này, chúng ta cần tiền xử lý văn bản, nhưng lần này chúng ta sẽ cắt nó thành các từ thay vì các ký tự. Để làm điều này, chúng ta có thể sử dụng lại lớp `tf.keras.layers.TextVectorization`. Lưu ý rằng nó sử dụng dấu cách để xác định ranh giới từ, điều này sẽ không hoạt động tốt trong một số ngôn ngữ. Ví dụ, chữ viết tiếng Trung không sử dụng dấu cách giữa các từ, tiếng Việt sử dụng dấu cách ngay cả trong các từ, và tiếng Đức thường gắn nhiều từ lại với nhau, không có dấu cách. Ngay cả trong tiếng Anh, dấu cách không phải lúc nào cũng là cách tốt nhất để phân tách văn bản: hãy nghĩ đến “San Francisco” hoặc “#ILoveDeepLearning”.

May mắn thay, có những giải pháp để giải quyết các vấn đề này. Trong một bài báo năm 2016, Rico Sennrich et al. từ Đại học Edinburgh đã khám phá một số phương pháp để phân tách và ghép lại văn bản ở cấp độ subword. Bằng cách này, ngay cả khi mô hình của bạn gặp một từ hiếm mà nó chưa từng thấy trước đây, nó vẫn có thể đoán được ý nghĩa của nó. Ví dụ, ngay cả khi mô hình chưa bao giờ thấy từ “smartest” trong quá trình huấn luyện, nếu nó học được từ “smart” và nó cũng học được rằng hậu tố “est” có nghĩa là “nhất”, nó có thể suy ra ý nghĩa của “smartest”. Một trong những kỹ thuật mà các tác giả đánh giá là mã hóa cặp byte (BPE). BPE hoạt động bằng cách chia toàn bộ tập huấn luyện thành các ký tự riêng lẻ (bao gồm cả dấu cách), sau đó liên tục hợp nhất các cặp liên kề thường xuyên nhất cho đến khi từ vựng đạt đến kích thước mong muốn.

Một bài báo năm 2018 của Taku Kudo tại Google đã cải thiện hơn nữa việc mã hóa subword, thường loại bỏ nhu cầu tiền xử lý cụ thể theo ngôn ngữ trước khi mã hóa. Hơn nữa, bài báo đã đề xuất một kỹ thuật chuẩn hóa mới được gọi là chuẩn hóa subword, giúp cải thiện độ chính xác và tính mạnh mẽ bằng cách đưa một số ngẫu nhiên vào việc mã hóa trong quá trình huấn luyện: ví dụ, “New England” có thể được mã hóa thành “New” + “England”, hoặc “New” + “Eng” + “land”, hoặc đơn giản là “New England” (chỉ một token). Dự án SentencePiece của Google cung cấp một triển khai mã nguồn mở, được mô tả trong một bài báo của Taku Kudo và John Richardson.

Thư viện TensorFlow Text cũng triển khai các chiến lược mã hóa khác nhau, bao gồm WordPiece (một biến thể của BPE), và cuối cùng nhưng không kém phần quan trọng, thư viện Tokenizers của Hugging Face triển khai một loạt các bộ mã hóa cực kỳ nhanh.

Tuy nhiên, đối với nhiệm vụ IMDb trong tiếng Anh, việc sử dụng dấu cách cho ranh giới từ sẽ là đủ tốt. Vì vậy, hãy tiếp tục tạo một lớp TextVectorization và điều chỉnh nó với tập huấn luyện. Chúng ta sẽ giới hạn từ vựng ở 1.000 token, bao gồm 998 từ thường xuyên nhất cộng với một token đệm và một token cho các từ không xác định, vì rất khó có khả năng các từ rất hiếm sẽ quan trọng cho nhiệm vụ này, và việc giới hạn kích thước từ vựng sẽ giảm số lượng tham số mà mô hình cần học:

```
vocab_size = 1000

text_vec_layer = tf.keras.layers.TextVectorization(max_tokens=vocab_size)
text_vec_layer.adapt(train_set.map(lambda reviews, labels: reviews))
```

Cuối cùng, chúng ta có thể tạo mô hình và huấn luyện nó:

```
embed_size = 128
tf.random.set_seed(42)
model = tf.keras.Sequential([
    text_vec_layer,
    tf.keras.layers.Embedding(vocab_size, embed_size),
    tf.keras.layers.GRU(128),
    tf.keras.layers.Dense(1, activation="sigmoid")
])
model.compile(loss="binary_crossentropy",
```

```
optimizer="nadam", metrics=["accuracy"])  
history = model.fit(train_set, validation_data=valid_set, epochs=2)
```

Lớp đầu tiên là lớp TextVectorization mà chúng ta vừa chuẩn bị, tiếp theo là một lớp Embedding sẽ chuyển đổi các ID từ thành các embedding. Ma trận embedding cần có một hàng cho mỗi token trong từ vựng (vocab_size) và một cột cho mỗi chiều embedding (ví dụ này sử dụng 128 chiều, nhưng đây là một siêu tham số bạn có thể điều chỉnh). Tiếp theo, chúng ta sử dụng một lớp GRU và một lớp Dense với một nơ-ron duy nhất và hàm kích hoạt sigmoid, vì đây là một nhiệm vụ phân loại nhị phân: đầu ra của mô hình sẽ là xác suất ước tính rằng đánh giá thể hiện một cảm xúc tích cực về bộ phim. Sau đó, chúng ta biên dịch mô hình, và chúng ta fit nó trên tập dữ liệu đã chuẩn bị trước đó trong vài epoch (hoặc bạn có thể huấn luyện lâu hơn để có kết quả tốt hơn).

Đáng buồn thay, nếu bạn chạy đoạn mã này, bạn thường sẽ thấy rằng mô hình không học được gì cả: độ chính xác vẫn gần 50%, không tốt hơn cơ hội ngẫu nhiên. Tại sao lại như vậy? Các đánh giá có độ dài khác nhau, vì vậy khi lớp TextVectorization chuyển đổi chúng thành chuỗi các ID token, nó sẽ đệm các chuỗi ngắn hơn bằng cách sử dụng token đệm (với ID 0) để làm cho chúng dài bằng chuỗi dài nhất trong batch. Kết quả là, hầu hết các chuỗi kết thúc bằng nhiều token đệm — thường là hàng chục hoặc thậm chí hàng trăm. Mặc dù chúng ta đang sử dụng một lớp GRU, tốt hơn nhiều so với lớp SimpleRNN, nhưng bộ nhớ ngắn hạn của nó vẫn không tốt, vì vậy khi nó đi qua nhiều token đệm, nó cuối cùng sẽ quên đánh giá đó là về cái gì! Một giải pháp là cung cấp cho mô hình các batch câu có độ dài bằng nhau (điều này cũng tăng tốc độ huấn luyện). Một giải pháp khác là làm cho RNN bỏ qua các token đệm. Điều này có thể được thực hiện bằng cách sử dụng mặt nạ (masking).

Mặt nạ (Masking)

Việc làm cho mô hình bỏ qua các token đệm là điều dễ dàng khi sử dụng Keras: chỉ cần thêm mask_zero=True khi tạo lớp Embedding. Điều này có nghĩa là các token đệm (có ID là 0) sẽ bị bỏ qua bởi tất cả các lớp phía sau. Chỉ vậy thôi! Nếu bạn huấn luyện lại mô hình trước đó trong vài epoch, bạn sẽ thấy rằng độ chính xác xác thực nhanh chóng đạt trên 80%.

Cách hoạt động của nó là lớp Embedding tạo một tensor mặt nạ bằng `tf.math.not_equal(inputs, 0)`: đó là một tensor Boolean có hình dạng giống như đầu vào, và nó bằng False ở bất cứ đâu các ID token là 0, hoặc True ở những nơi khác. Tensor mặt nạ này sau đó được mô hình tự động lan truyền đến lớp tiếp theo. Nếu phương thức `call()` của lớp đó có một đối số mask, thì nó sẽ tự động nhận mặt nạ. Điều này cho phép lớp bỏ qua các bước thời gian thích hợp. Mỗi lớp có thể xử lý mặt nạ khác nhau, nhưng nói chung chúng chỉ đơn giản bỏ qua các bước thời gian được che (tức là các bước thời gian mà mặt nạ là False).

Ví dụ, khi một lớp hồi quy gặp một bước thời gian được che, nó chỉ đơn giản sao chép đầu ra từ bước thời gian trước đó.

Tiếp theo, nếu thuộc tính `supports_masking` của lớp là True, thì mặt nạ sẽ tự động lan truyền đến lớp tiếp theo. Nó tiếp tục lan truyền theo cách này miễn là các lớp có `supports_masking=True`.

Ví dụ, thuộc tính `supports_masking` của lớp hồi quy là `True` khi `return_sequences=True`, nhưng nó là `False` khi `return_sequences=False` vì không cần mặt nạ nữa trong trường hợp này. Vì vậy, nếu bạn có một mô hình với một số lớp hồi quy với `return_sequences=True`, theo sau là một lớp hồi quy với `return_sequences=False`, thì mặt nạ sẽ tự động lan truyền đến lớp hồi quy cuối cùng: lớp đó sẽ sử dụng mặt nạ để bỏ qua các bước được che, nhưng nó sẽ không lan truyền mặt nạ đi xa hơn.

Tương tự, nếu bạn đặt `mask_zero=True` khi tạo lớp `Embedding` trong mô hình phân tích cảm xúc mà chúng ta vừa xây dựng, thì lớp `GRU` sẽ tự động nhận và sử dụng mặt nạ, nhưng nó sẽ không lan truyền mặt nạ đi xa hơn, vì `return_sequences` không được đặt thành `True`.

Nhiều lớp Keras hỗ trợ mặt nạ: `SimpleRNN`, `GRU`, `LSTM`, `Bidirectional`, `Dense`, `TimeDistributed`, `Add`, và một vài lớp khác (tất cả trong gói `tf.keras.layers`). Tuy nhiên, các lớp tích chập (bao gồm `Conv1D`) không hỗ trợ mặt nạ — dù sao thì không rõ chúng sẽ làm điều đó như thế nào.

Nếu mặt nạ lan truyền đến tận đầu ra, thì nó cũng được áp dụng cho các hàm mất mát, vì vậy các bước thời gian được che sẽ không đóng góp vào mất mát (mất mát của chúng sẽ là 0). Điều này giả định rằng mô hình xuất ra các chuỗi, điều này không đúng trong mô hình phân tích cảm xúc của chúng ta.

Nếu bạn muốn triển khai lớp tùy chỉnh của riêng mình với hỗ trợ mặt nạ, bạn nên thêm một đối số `mask` vào phương thức `call()`, và rõ ràng làm cho phương thức sử dụng mặt nạ. Ngoài ra, nếu mặt nạ phải được lan truyền đến các lớp tiếp theo, thì bạn nên đặt `self.supports_masking=True` trong hàm tạo. Nếu mặt nạ phải được cập nhật trước khi nó được lan truyền, thì bạn phải triển khai phương thức `compute_mask()`.

Nếu mô hình của bạn không bắt đầu bằng một lớp `Embedding`, bạn có thể sử dụng lớp `tf.keras.layers.Masking` thay thế: theo mặc định, nó đặt mặt nạ thành `tf.math.reduce_any(tf.math.not_equal(X, 0), axis=-1)`, nghĩa là các bước thời gian mà chiều cuối cùng chứa đầy các số 0 sẽ bị che trong các lớp tiếp theo.

Sử dụng các lớp mặt nạ và lan truyền mặt nạ tự động hoạt động tốt nhất cho các mô hình đơn giản. Nó sẽ không phải lúc nào cũng hoạt động cho các mô hình phức tạp hơn, chẳng hạn như khi bạn cần trộn các lớp `Conv1D` với các lớp hồi quy. Trong những trường hợp như vậy, bạn sẽ cần tính toán mặt nạ một cách rõ ràng và truyền nó đến các lớp thích hợp, sử dụng API chức năng hoặc API phân lớp. Ví dụ, mô hình sau đây tương đương với mô hình trước đó, ngoại trừ nó được xây dựng bằng API chức năng và xử lý mặt nạ thủ công. Nó cũng thêm một chút dropout vì mô hình trước đó hơi bị overfitting:

```
inputs = tf.keras.layers.Input(shape=[], dtype=tf.string)
token_ids = text_vec_layer(inputs)
mask = tf.math.not_equal(token_ids, 0)
Z = tf.keras.layers.Embedding(vocab_size, embed_size)(token_ids)
Z = tf.keras.layers.GRU(128, dropout=0.2)(Z, mask=mask)
outputs = tf.keras.layers.Dense(1, activation="sigmoid")(Z)
model = tf.keras.Model(inputs=[inputs], outputs=[outputs])
```

Một cách tiếp cận cuối cùng để che mặt là cung cấp cho mô hình các tensor lờm chờm (ragged tensors). Trên thực tế, tất cả những gì bạn cần làm là đặt `ragged=True` khi tạo lớp `TextVectorization`, để các chuỗi đầu vào được biểu diễn dưới dạng các tensor lờm chờm:

```
>>> text_vec_layer_ragged = tf.keras.layers.TextVectorization(
...     max_tokens=vocab_size, ragged=True)
...
>>> text_vec_layer_ragged.adapt(train_set.map(lambda reviews, labels: reviews
...))
>>> text_vec_layer_ragged(["Great movie!", "This is DiCaprio's best role."])
<tf.RaggedTensor [[86, 18], [11, 7, 1, 116, 217]]>
```

Số sánh biểu diễn tensor lờm chờm này với biểu diễn tensor thông thường, sử dụng các token đệm:

```
>>> text_vec_layer(["Great movie!", "This is DiCaprio's best role."])
<tf.Tensor:
shape=(2, 5), dtype=int64, numpy= array([[ 86,18,          0,  0,  0],
[ 11,  7,  1, 116, 217]])>
```

Các lớp hồi quy của Keras có hỗ trợ tích hợp cho các tensor lờm chờm, vì vậy bạn không cần làm gì khác: chỉ cần sử dụng lớp `TextVectorization` này trong mô hình của bạn. Không cần truyền `mask_zero=True` hoặc xử lý mặt nạ một cách rõ ràng — tất cả đều được triển khai cho bạn. Điều đó thật tiện lợi! Tuy nhiên, tính đến đầu năm 2022, hỗ trợ cho các tensor lờm chờm trong Keras vẫn còn khá mới, vì vậy vẫn còn một vài khuyết điểm. Ví dụ, hiện tại không thể sử dụng các tensor lờm chờm làm mục tiêu khi chạy trên GPU (nhưng điều này có thể được giải quyết vào thời điểm bạn đọc những dòng này).

Bất kể phương pháp che mặt nào bạn thích, sau khi huấn luyện mô hình này trong vài epoch, nó sẽ trở nên khá tốt trong việc đánh giá một đánh giá là tích cực hay không. Nếu bạn sử dụng callback `tf.keras.callbacks.TensorBoard()`, bạn có thể trực quan hóa các embedding trong TensorBoard khi chúng đang được học: thật thú vị khi thấy các từ như “awesome” và “amazing” dần dần tập hợp ở một phía của không gian embedding, trong khi các từ như “awful” và “terrible” tập hợp ở phía bên kia. Một số từ không tích cực như bạn mong đợi (ít nhất là với mô hình này), chẳng hạn như từ “good”, có lẽ vì nhiều đánh giá tiêu cực chứa cụm từ “not good”.

Tái sử dụng Embedding và Mô hình ngôn ngữ đã được huấn luyện trước

Thật ấn tượng khi mô hình có thể học được các embedding từ có ích chỉ dựa trên 25.000 đánh giá phim. Hãy tưởng tượng các embedding sẽ tốt đến mức nào nếu chúng ta có hàng tỷ đánh giá để huấn luyện!

Thật không may, chúng ta không có, nhưng có lẽ chúng ta có thể tái sử dụng các embedding từ đã được huấn luyện trên một kho văn bản (rất) lớn khác (ví dụ: đánh giá của Amazon, có sẵn trên TensorFlow Datasets), ngay cả khi nó không bao gồm các đánh giá phim? Dù sao đi nữa, từ “amazing” thường có cùng ý nghĩa dù bạn sử dụng nó để nói về phim hay bất cứ điều gì khác.

Hơn nữa, có lẽ các embedding sẽ hữu ích cho phân tích cảm xúc ngay cả khi chúng được huấn luyện cho một nhiệm vụ khác: vì các từ như “awesome” và “amazing” có ý nghĩa tương tự, chúng có khả năng tập hợp trong không gian embedding ngay cả đối với các nhiệm vụ như dự đoán từ tiếp theo trong câu. Nếu tất cả các từ tích cực và tất cả các từ tiêu cực tạo thành các cụm, thì điều này sẽ hữu ích cho phân tích cảm xúc. Vì vậy, thay vì huấn luyện các embedding từ, chúng ta có thể tải xuống và sử dụng các embedding đã được huấn luyện trước, chẳng hạn như embedding Word2vec của Google, embedding GloVe của Stanford hoặc embedding FastText của Facebook.

Sử dụng các embedding từ đã được huấn luyện trước đã phổ biến trong vài năm, nhưng cách tiếp cận này có những giới hạn. Đặc biệt, một từ có một biểu diễn duy nhất, bất kể ngữ cảnh. Ví dụ, từ “right” được mã hóa giống nhau trong “left and right” và “right and wrong”, mặc dù nó có nghĩa là hai điều rất khác nhau. Để giải quyết hạn chế này, một bài báo năm 2018 của Matthew Peters đã giới thiệu Embedding from Language Models (ELMo): đây là các embedding từ theo ngữ cảnh được học từ các trạng thái bên trong của một mô hình ngôn ngữ hai chiều sâu. Thay vì chỉ sử dụng các embedding đã được huấn luyện trước trong mô hình của bạn, bạn tái sử dụng một phần của mô hình ngôn ngữ đã được huấn luyện trước.

Khoảng cùng thời gian, bài báo Universal Language Model Fine-Tuning (ULMFiT) của Jeremy Howard và Sebastian Ruder đã chứng minh hiệu quả của việc tiền huấn luyện không giám sát cho các nhiệm vụ NLP: các tác giả đã huấn luyện một mô hình ngôn ngữ LSTM trên một kho văn bản khổng lồ bằng cách sử dụng học tự giám sát (tức là tự động tạo nhãn từ dữ liệu), sau đó họ tinh chỉnh nó trên các nhiệm vụ khác nhau. Mô hình của họ vượt trội so với các mô hình tiên tiến nhất trên sáu nhiệm vụ phân loại văn bản với biên độ lớn (giảm tỷ lệ lỗi 18–24% trong hầu hết các trường hợp). Hơn nữa, các tác giả đã chỉ ra rằng một mô hình được huấn luyện trước được tinh chỉnh chỉ với 100 ví dụ được gán nhãn có thể đạt được hiệu suất tương tự như một mô hình được huấn luyện từ đầu trên 10.000 ví dụ. Trước bài báo ULMFiT, việc sử dụng các mô hình được huấn luyện trước chỉ là chuẩn mực trong thị giác máy tính; trong bối cảnh NLP, việc tiền huấn luyện chỉ giới hạn ở các embedding từ. Bài báo này đã đánh dấu sự khởi đầu của một kỷ nguyên mới trong NLP: ngày nay, việc tái sử dụng các mô hình ngôn ngữ được huấn luyện trước là chuẩn mực.

Ví dụ, hãy xây dựng một bộ phân loại dựa trên Universal Sentence Encoder, một kiến trúc mô hình được giới thiệu trong một bài báo năm 2018 của một nhóm các nhà nghiên cứu Google. Mô hình này dựa trên kiến trúc transformer, mà chúng ta sẽ xem xét sau trong chương này. Tiện lợi thay, mô hình này có sẵn trên TensorFlow Hub:

```
import tensorflow_hub as hub
import os

os.environ["TFHUB_CACHE_DIR"] = "my_tfhub_cache"
model = tf.keras.Sequential([
    hub.KerasLayer("https://tfhub.dev/google/universal-sentence-encoder/4",
                  trainable=True, dtype=tf.string, input_shape=[]),
    tf.keras.layers.Dense(64, activation="relu"),
    tf.keras.layers.Dense(1, activation="sigmoid")
])
```

```
])  
model.compile(loss="binary_crossentropy",  
              optimizer="nadam", metrics=["accuracy"])  
model.fit(train_set, validation_data=valid_set, epochs=10)
```

Lưu ý rằng phần cuối cùng của URL mô-đun TensorFlow Hub chỉ định rằng chúng ta muốn phiên bản 4 của mô hình. Việc đánh phiên bản này đảm bảo rằng nếu một phiên bản mô-đun mới được phát hành trên TF Hub, nó sẽ không phá vỡ mô hình của chúng ta. Tiện lợi thay, nếu bạn chỉ cần nhập URL này vào trình duyệt web, bạn sẽ nhận được tài liệu cho mô-đun này.

Cũng lưu ý rằng chúng ta đặt `trainable=True` khi tạo `hub.KerasLayer`. Bằng cách này, Universal Sentence Encoder đã được huấn luyện trước được tinh chỉnh trong quá trình huấn luyện: một số trọng số của nó được điều chỉnh thông qua lan truyền ngược. Không phải tất cả các mô-đun TensorFlow Hub đều có thể tinh chỉnh, vì vậy hãy đảm bảo kiểm tra tài liệu cho từng mô-đun đã được huấn luyện trước mà bạn quan tâm.

Sau khi huấn luyện, mô hình này sẽ đạt độ chính xác thực trên 90%. Điều đó thực sự rất tốt: nếu bạn cố gắng tự thực hiện nhiệm vụ, bạn có thể chỉ làm tốt hơn một chút vì nhiều đánh giá chứa cả bình luận tích cực và tiêu cực. Phân loại các đánh giá mơ hồ này giống như tung đồng xu.

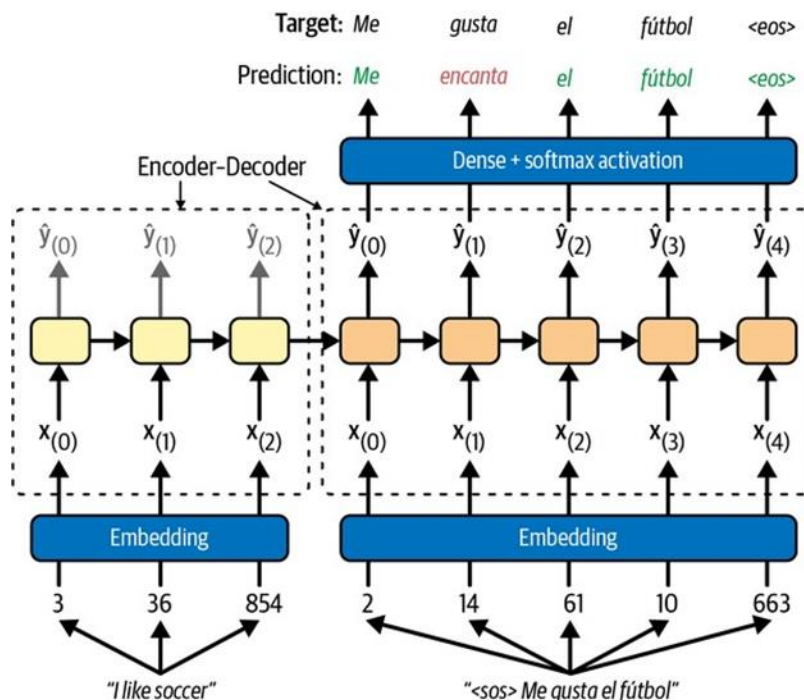
Cho đến nay, chúng ta đã xem xét việc tạo văn bản bằng char-RNN và phân tích cảm xúc bằng các mô hình RNN cấp từ (dựa trên các embedding có thể huấn luyện) và sử dụng một mô hình ngôn ngữ mạnh mẽ đã được huấn luyện trước từ TensorFlow Hub. Trong phần tiếp theo, chúng ta sẽ khám phá một nhiệm vụ NLP quan trọng khác: dịch máy thần kinh (NMT).

Mạng mã hóa-giải mã cho dịch máy thần kinh

Hãy bắt đầu với một mô hình NMT đơn giản sẽ dịch các câu tiếng Anh sang tiếng Tây Ban Nha (xem Hình 16-3). Tóm lại, kiến trúc như sau: các câu tiếng Anh được đưa vào làm đầu vào cho bộ mã hóa, và bộ giải mã xuất ra các bản dịch tiếng Tây Ban Nha. Lưu ý rằng các bản dịch tiếng Tây Ban Nha cũng được sử dụng làm đầu vào cho bộ giải mã trong quá trình huấn luyện, nhưng được dịch chuyển lùi một bước. Nói cách khác, trong quá trình huấn luyện, bộ giải mã được cung cấp làm đầu vào từ mà nó đáng lẽ phải xuất ra ở bước trước đó, bất kể nó thực sự xuất ra gì. Đây được gọi là *teacher forcing* — một kỹ thuật giúp tăng tốc đáng kể việc huấn luyện và cải thiện hiệu suất của mô hình. Đối với từ đầu tiên, bộ giải mã được cung cấp token bắt đầu chuỗi (SOS), và bộ giải mã được kỳ vọng kết thúc câu bằng token kết thúc chuỗi (EOS). Mỗi từ ban đầu được biểu diễn bằng ID của nó (ví dụ: 854 cho từ “soccer”). Tiếp theo, một lớp Embedding trả về embedding của từ. Các embedding từ này sau đó được đưa vào bộ mã hóa và bộ giải mã.

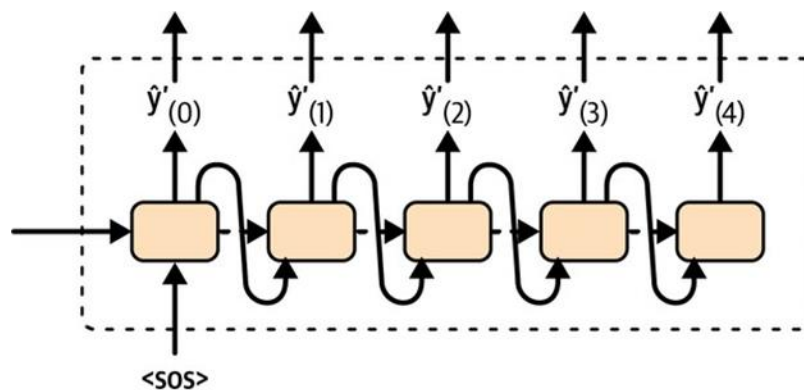
Ở mỗi bước, bộ giải mã xuất ra một điểm cho mỗi từ trong từ vựng đầu ra (tức là tiếng Tây Ban Nha), sau đó hàm kích hoạt softmax biến các điểm này thành xác suất. Ví dụ, ở bước đầu tiên, từ “Me” có thể có xác suất 7%, “Yo” có thể có xác suất 1%, v.v. Từ có xác suất cao nhất được xuất ra. Điều này rất giống với một nhiệm vụ phân loại thông thường, và thực sự

bạn có thể huấn luyện mô hình bằng cách sử dụng hàm mất mát "sparse_categorical_crossentropy", giống như chúng ta đã làm trong mô hình char-RNN.



Hình 16-3. Một mô hình dịch máy đơn giản

Lưu ý rằng tại thời điểm suy luận (sau khi huấn luyện), bạn sẽ không có câu mục tiêu để đưa vào bộ giải mã. Thay vào đó, bạn cần đưa vào từ mà nó vừa xuất ra ở bước trước đó, như trong Hình 16-4 (điều này sẽ yêu cầu tra cứu embedding không được hiển thị trong sơ đồ).



Hình 16-4. Tại thời điểm suy luận, bộ giải mã được cung cấp làm đầu vào từ mà nó vừa xuất ra ở bước thời gian trước đó

Hãy xây dựng và huấn luyện mô hình này! Đầu tiên, chúng ta cần tải xuống một tập dữ liệu các cặp câu tiếng Anh/Tây Ban Nha:

```
import tensorflow as tf
from pathlib import Path
```

```
import numpy as np

url = "https://storage.googleapis.com/download.tensorflow.org/data/spa-eng.zip"
path = tf.keras.utils.get_file("spa-eng.zip", origin=url, cache_dir="datasets",
                                extract=True)
text = (Path(path).with_name("spa-eng") / "spa.txt").read_text()
```

Mỗi dòng chứa một câu tiếng Anh và bản dịch tiếng Tây Ban Nha tương ứng, được phân tách bằng một tab. Chúng ta sẽ bắt đầu bằng cách loại bỏ các ký tự tiếng Tây Ban Nha “ı” và “¿”, mà lớp TextVectorization không xử lý, sau đó chúng ta sẽ phân tích cú pháp các cặp câu và xáo trộn chúng. Cuối cùng, chúng ta sẽ chia chúng thành hai danh sách riêng biệt, mỗi danh sách một ngôn ngữ:

```
text = text.replace("ı", "").replace("¿", "")

pairs = [line.split("\t") for line in text.splitlines()]
np.random.shuffle(pairs)
sentences_en, sentences_es = zip(*pairs) # tách các cặp thành 2 danh sách
```

Hãy xem xét ba cặp câu đầu tiên:

```
>>> for i in range(3):
...     print(sentences_en[i], "=>", sentences_es[i])
...
How boring! => Qué aburrimiento!
I love sports. => Adoro el deporte.
Would you like to swap jobs? => Te gustaría que intercambiamos los trabajos?
```

Tiếp theo, hãy tạo hai lớp TextVectorization — một cho mỗi ngôn ngữ — và điều chỉnh chúng vào văn bản:

```
vocab_size = 1000
max_length = 50
text_vec_layer_en = tf.keras.layers.TextVectorization(
    vocab_size, output_sequence_length=max_length)
text_vec_layer_es = tf.keras.layers.TextVectorization(
    vocab_size, output_sequence_length=max_length)
text_vec_layer_en.adapt(sentences_en)

text_vec_layer_es.adapt([f"startofseq {s} endofseq" for s in sentences_es])
```

Có một vài điều cần lưu ý ở đây:

- Chúng ta giới hạn kích thước từ vựng ở 1.000, khá nhỏ. Điều này là do tập huấn luyện không quá lớn, và vì sử dụng một giá trị nhỏ sẽ tăng tốc độ huấn luyện. Các mô hình dịch thuật tiên tiến thường sử dụng từ vựng lớn hơn nhiều (ví dụ: 30.000), tập huấn luyện lớn hơn nhiều (gigabyte), và mô hình lớn hơn nhiều (hàng trăm hoặc thậm chí

hàng nghìn megabyte). Ví dụ, hãy xem các mô hình Opus-MT của Đại học Helsinki, hoặc mô hình M2M-100 của Facebook.

- Vì tất cả các câu trong tập dữ liệu có tối đa 50 từ, chúng ta đặt `output_sequence_length` thành 50: bằng cách này, các chuỗi đầu vào sẽ tự động được đệm bằng các số 0 cho đến khi chúng đều dài 50 token. Nếu có bất kỳ câu nào dài hơn 50 token trong tập huấn luyện, nó sẽ được cắt thành 50 token.
- Đối với văn bản tiếng Tây Ban Nha, chúng ta thêm “startofseq” và “endofseq” vào mỗi câu khi điều chỉnh lớp `TextVectorization`: chúng ta sẽ sử dụng các từ này làm token SOS và EOS. Bạn có thể sử dụng bất kỳ từ nào khác, miễn là chúng không phải là các từ tiếng Tây Ban Nha thực tế.

Hãy kiểm tra 10 token đầu tiên trong cả hai từ vựng. Chúng bắt đầu bằng token đệm, token không xác định, token SOS và EOS (chỉ trong từ vựng tiếng Tây Ban Nha), sau đó là các từ thực tế, được sắp xếp theo tần suất giảm dần:

```
>>> text_vec_layer_en.get_vocabulary()[:10]
['', '[UNK]', 'the', 'i', 'to', 'you', 'tom', 'a', 'is', 'he']

>>> text_vec_layer_es.get_vocabulary()[:10]
['', '[UNK]', 'startofseq', 'endofseq', 'de', 'que', 'a', 'no', 'tom', 'la']
```

Tiếp theo, hãy tạo tập huấn luyện và tập xác thực (bạn cũng có thể tạo tập kiểm tra nếu bạn cần). Chúng ta sẽ sử dụng 100.000 cặp câu đầu tiên để huấn luyện, và phần còn lại để xác thực. Đầu vào của bộ giải mã là các câu tiếng Tây Ban Nha cộng với tiền tố token SOS. Các mục tiêu là các câu tiếng Tây Ban Nha cộng với hậu tố EOS:

```
X_train = tf.constant(sentences_en[:100_000])
X_valid = tf.constant(sentences_en[100_000:])
X_train_dec = tf.constant([f"startofseq {s}" for s in sentences_es[:100_000]]
)
X_valid_dec = tf.constant([f"startofseq {s}" for s in sentences_es[100_000:]]
)
Y_train = text_vec_layer_es([f"{s} endofseq" for s in sentences_es[:100_000]]
)
Y_valid = text_vec_layer_es([f"{s} endofseq" for s in sentences_es[100_000:]]
)
```

OK, bây giờ chúng ta đã sẵn sàng xây dựng mô hình dịch thuật của mình. Chúng ta sẽ sử dụng API chức năng cho việc đó vì mô hình không phải là tuần tự. Nó yêu cầu hai đầu vào văn bản — một cho bộ mã hóa và một cho bộ giải mã — vì vậy hãy bắt đầu với điều đó:

```
encoder_inputs = tf.keras.layers.Input(shape=[], dtype=tf.string)
decoder_inputs = tf.keras.layers.Input(shape=[], dtype=tf.string)
```

Tiếp theo, chúng ta cần mã hóa các câu này bằng cách sử dụng các lớp `TextVectorization` mà chúng ta đã chuẩn bị trước đó, tiếp theo là một lớp `Embedding` cho mỗi ngôn ngữ, với `mask_zero=True` để đảm bảo che mặt được xử lý tự động. Kích thước embedding là một siêu tham số bạn có thể điều chỉnh, như mọi khi:

```

embed_size = 128
encoder_input_ids = text_vec_layer_en(encoder_inputs)
decoder_input_ids = text_vec_layer_es(decoder_inputs)
encoder_embedding_layer = tf.keras.layers.Embedding(vocab_size, embed_size,
                                                    mask_zero=True)
decoder_embedding_layer = tf.keras.layers.Embedding(vocab_size, embed_size,
                                                    mask_zero=True)
encoder_embeddings = encoder_embedding_layer(encoder_input_ids)
decoder_embeddings = decoder_embedding_layer(decoder_input_ids)

```

Bây giờ hãy tạo bộ mã hóa và truyền các đầu vào đã được nhúng vào nó:

```

encoder = tf.keras.layers.LSTM(512, return_state=True)
encoder_outputs, *encoder_state = encoder(encoder_embeddings)

```

Để giữ mọi thứ đơn giản, chúng ta chỉ sử dụng một lớp LSTM duy nhất, nhưng bạn có thể xếp chồng nhiều lớp. Chúng ta cũng đặt `return_state=True` để có một tham chiếu đến trạng thái cuối cùng của lớp. Vì chúng ta đang sử dụng lớp LSTM, thực sự có hai trạng thái: trạng thái ngắn hạn và trạng thái dài hạn. Lớp trả về các trạng thái này riêng biệt, đó là lý do tại sao chúng ta phải viết `*encoder_state` để nhóm cả hai trạng thái vào một danh sách. Bây giờ chúng ta có thể sử dụng trạng thái (kép) này làm trạng thái ban đầu của bộ giải mã:

```

decoder = tf.keras.layers.LSTM(512, return_sequences=True)
decoder_outputs = decoder(decoder_embeddings, initial_state=encoder_state)

```

Tiếp theo, chúng ta có thể truyền đầu ra của bộ giải mã qua một lớp Dense với hàm kích hoạt softmax để lấy xác suất từ cho mỗi bước:

```

output_layer = tf.keras.layers.Dense(vocab_size, activation="softmax")
Y_proba = output_layer(decoder_outputs)

```

Và thế là xong! Chúng ta chỉ cần tạo Keras Model, biên dịch nó và huấn luyện nó:

```

model = tf.keras.Model(inputs=[encoder_inputs, decoder_inputs],
                      outputs=[Y_proba])
model.compile(loss="sparse_categorical_crossentropy",
              optimizer="nadam", metrics=["accuracy"])

model.fit((X_train, X_train_dec), Y_train, epochs=10,
        validation_data=((X_valid, X_valid_dec), Y_valid))

```

Sau khi huấn luyện, chúng ta có thể sử dụng mô hình để dịch các câu tiếng Anh mới sang tiếng Tây Ban Nha. Nhưng nó không đơn giản như gọi `model.predict()`, bởi vì bộ giải mã mong đợi đầu vào là từ đã được dự đoán ở bước thời gian trước đó. Một cách để làm điều này là viết một ô nhớ tùy chỉnh theo dõi đầu ra trước đó và truyền nó đến bộ mã hóa ở bước thời gian tiếp theo. Tuy nhiên, để giữ mọi thứ đơn giản, chúng ta có thể chỉ cần gọi mô hình nhiều lần, dự đoán thêm một từ ở mỗi vòng. Hãy viết một hàm tiện ích nhỏ cho việc đó:

```

def translate(sentence_en):
    translation = ""

```

```

    for word_idx in range(max_length):
        X = np.array([sentence_en]) # đầu vào bộ mã hóa
        X_dec = np.array(["startofseq " + translation]) # đầu vào bộ giải mã
        y_proba = model.predict((X, X_dec))[0, word_idx] # xác suất của token
cuối cùng
        predicted_word_id = np.argmax(y_proba)

        predicted_word = text_vec_layer_es.get_vocabulary()[predicted_word_id
]
        if predicted_word == "endofseq":
            break
        translation += " " + predicted_word
    return translation.strip()

```

Hàm này chỉ đơn giản là tiếp tục dự đoán từng từ một, dần dần hoàn thành bản dịch, và nó dừng lại khi đạt đến token EOS. Hãy thử xem!

```

>>> translate("I
like soccer") 'me gusta el fútbol'

```

Hoan hô, nó hoạt động! Chà, ít nhất là với những câu rất ngắn. Nếu bạn thử chơi với mô hình này một lúc, bạn sẽ thấy rằng nó chưa song ngữ, và đặc biệt là nó thực sự gặp khó khăn với những câu dài hơn. Ví dụ:

```

>>> translate("I
like soccer and also going to the beach") 'me gusta el fútbol y a
veces mismo al bus'

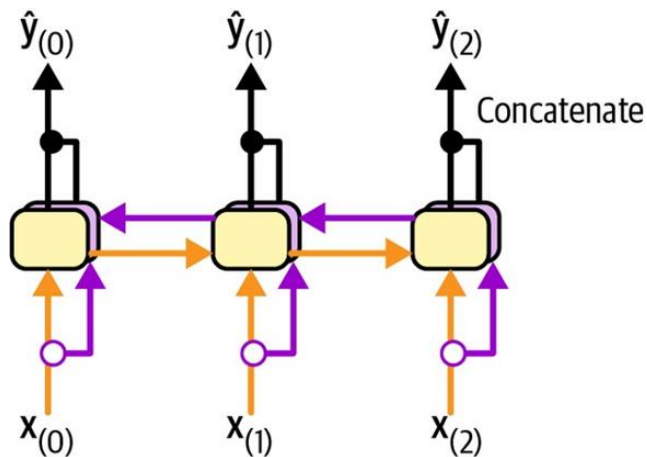
```

Bản dịch nói “Tôi thích bóng đá và đôi khi cả xe buýt”. Vậy làm thế nào bạn có thể cải thiện nó? Một cách là tăng kích thước tập huấn luyện và thêm nhiều lớp LSTM hơn trong cả bộ mã hóa và bộ giải mã. Nhưng điều này sẽ chỉ giúp bạn đến một mức độ nào đó, vì vậy hãy xem xét các kỹ thuật phức tạp hơn, bắt đầu với các lớp hồi quy hai chiều.

RNN hai chiều

Ở mỗi bước thời gian, một lớp hồi quy thông thường chỉ xem xét các đầu vào trong quá khứ và hiện tại trước khi tạo ra đầu ra của nó. Nói cách khác, nó mang tính nhân quả, nghĩa là nó không thể nhìn thấy tương lai. Loại RNN này có ý nghĩa khi dự báo chuỗi thời gian, hoặc trong bộ giải mã của một mô hình tuần tự (seq2seq). Nhưng đối với các nhiệm vụ như phân loại văn bản, hoặc trong bộ mã hóa của một mô hình seq2seq, thường thì nên nhìn trước các từ tiếp theo trước khi mã hóa một từ đã cho.

Ví dụ, hãy xem xét các cụm từ “the right arm” (cánh tay phải), “the right person” (người phù hợp), và “the right to criticize” (quyền chỉ trích): để mã hóa đúng từ “right”, bạn cần nhìn về phía trước. Một giải pháp là chạy hai lớp hồi quy trên cùng một đầu vào, một lớp đọc các từ từ trái sang phải và lớp còn lại đọc chúng từ phải sang trái, sau đó kết hợp các đầu ra của chúng ở mỗi bước thời gian, thường là bằng cách nối chúng lại. Đây là những gì một lớp hồi quy hai chiều thực hiện (xem Hình 16-5).



Hình 16-5. Một lớp hồi quy hai chiều

Để triển khai một lớp hồi quy hai chiều trong Keras, chỉ cần gói một lớp hồi quy trong một lớp `tf.keras.layers.Bidirectional`. Ví dụ, lớp `Bidirectional` sau đây có thể được sử dụng làm bộ mã hóa trong mô hình dịch thuật của chúng ta:

```
encoder = tf.keras.layers.Bidirectional(
    tf.keras.layers.LSTM(256, return_sequences=True, return_state=True))
```

Chỉ có một vấn đề. Lớp này bây giờ sẽ trả về bốn trạng thái thay vì hai: các trạng thái ngắn hạn và dài hạn cuối cùng của lớp LSTM chuyển tiếp, và các trạng thái ngắn hạn và dài hạn cuối cùng của lớp LSTM ngược. Chúng ta không thể sử dụng trạng thái bốn này trực tiếp làm trạng thái ban đầu của lớp LSTM của bộ giải mã, vì nó chỉ mong đợi hai trạng thái (ngắn hạn và dài hạn). Chúng ta không thể làm cho bộ giải mã hai chiều, vì nó phải giữ tính nhân quả: nếu không nó sẽ gian lận trong quá trình huấn luyện và nó sẽ không hoạt động. Thay vào đó, chúng ta có thể nối hai trạng thái ngắn hạn lại với nhau, và cũng nối hai trạng thái dài hạn lại với nhau:

```
encoder_outputs, *encoder_state = encoder(encoder_embeddings)
encoder_state = [tf.concat(encoder_state[:, :2], axis=-1), # ngắn hạn (0 & 2)
                 tf.concat(encoder_state[1:3], axis=-1)] # dài hạn (1 & 3)
```

Bây giờ chúng ta hãy xem một kỹ thuật phổ biến khác có thể cải thiện đáng kể hiệu suất của mô hình dịch thuật tại thời điểm suy luận: tìm kiếm chùm (beam search).

Tìm kiếm chùm (Beam Search)

Giả sử bạn đã huấn luyện một mô hình mã hóa-giải mã, và bạn sử dụng nó để dịch câu “I like soccer” sang tiếng Tây Ban Nha. Bạn đang hy vọng rằng nó sẽ xuất ra bản dịch chính xác “me gusta el fútbol”, nhưng thật không may nó lại xuất ra “me gustan los jugadores”, có nghĩa là “Tôi thích các cầu thủ”. Nhìn vào tập huấn luyện, bạn nhận thấy nhiều câu như “I like cars”, được dịch là “me gustan los autos”, vì vậy không phải là vô lý khi mô hình xuất ra “me gustan los” sau khi thấy “I like”. Thật không may, trong trường hợp này đó là một lỗi vì “soccer” là số ít. Mô hình không thể quay lại và sửa lỗi, vì vậy nó cố gắng hoàn thành câu tốt nhất có thể, trong trường hợp này sử dụng từ “jugadores”. Làm thế nào chúng ta có thể cho mô hình cơ

hội quay lại và sửa lỗi mà nó đã mắc phải trước đó? Một trong những giải pháp phổ biến nhất là tìm kiếm chùm: nó theo dõi một danh sách ngắn gồm k câu hứa hẹn nhất (ví dụ, ba câu hàng đầu), và ở mỗi bước giải mã, nó cố gắng mở rộng chúng thêm một từ, chỉ giữ lại k câu có khả năng nhất. Tham số k được gọi là chiều rộng chùm (beam width).

Ví dụ, giả sử bạn sử dụng mô hình để dịch câu “I like soccer” bằng cách sử dụng tìm kiếm chùm với chiều rộng chùm là 3 (xem Hình 16-6). Ở bước giải mã đầu tiên, mô hình sẽ xuất ra xác suất ước tính cho mỗi từ đầu tiên có thể có trong câu được dịch. Giả sử ba từ hàng đầu là “me” (xác suất ước tính 75%), “a” (3%), và “como” (1%). Đó là danh sách ngắn của chúng ta cho đến nay. Tiếp theo, chúng ta sử dụng mô hình để tìm từ tiếp theo cho mỗi câu. Đối với câu đầu tiên (“me”), có lẽ mô hình xuất ra xác suất 36% cho từ “gustan”, 32% cho từ “gusta”, 16% cho từ “encanta”, v.v. Lưu ý rằng đây thực sự là các xác suất có điều kiện, với điều kiện câu bắt đầu bằng “me”. Đối với câu thứ hai (“a”), mô hình có thể xuất ra xác suất có điều kiện 50% cho từ “mi”, v.v. Giả sử từ vựng có 1.000 từ, chúng ta sẽ có 1.000 xác suất cho mỗi câu.

Tiếp theo, chúng ta tính xác suất của mỗi trong số 3.000 câu hai từ mà chúng ta đã xem xét (3×1.000). Chúng ta làm điều này bằng cách nhân xác suất có điều kiện ước tính của mỗi từ với xác suất ước tính của câu mà nó hoàn thành. Ví dụ, xác suất ước tính của câu “me” là 75%, trong khi xác suất có điều kiện ước tính của từ “gustan” (với điều kiện từ đầu tiên là “me”) là 36%, vì vậy xác suất ước tính của câu “me gustan” là $75\% \times 36\% = 27\%$. Sau khi tính toán xác suất của tất cả 3.000 câu hai từ, chúng ta chỉ giữ lại 3 câu hàng đầu. Trong ví dụ này, tất cả chúng đều bắt đầu bằng từ “me”: “me gustan” (27%), “me gusta” (24%), và “me encanta” (12%).

Hiện tại, câu “me gustan” đang thắng, nhưng “me gusta” chưa bị loại bỏ.



Hình 16-6. Tìm kiếm chùm, với chiều rộng chùm là 3

Sau đó, chúng ta lặp lại cùng một quá trình: chúng ta sử dụng mô hình để dự đoán từ tiếp theo trong mỗi ba câu này, và chúng ta tính toán xác suất của tất cả 3.000 câu ba từ mà chúng ta đã xem xét. Có lẽ ba câu hàng đầu hiện nay là “me gustan los” (10%), “me gusta el” (8%), và “me gusta mucho” (2%). Ở bước tiếp theo, chúng ta có thể nhận được “me gusta

el fútbol” (6%), “me gusta mucho el” (1%), và “me gusta el deporte” (0,2%). Lưu ý rằng “me gustan” đã bị loại bỏ, và bản dịch đúng hiện đang dẫn trước. Chúng ta đã tăng hiệu suất của mô hình mã hóa-giải mã của mình mà không cần huấn luyện thêm, chỉ đơn giản bằng cách sử dụng nó một cách khôn ngoan hơn.

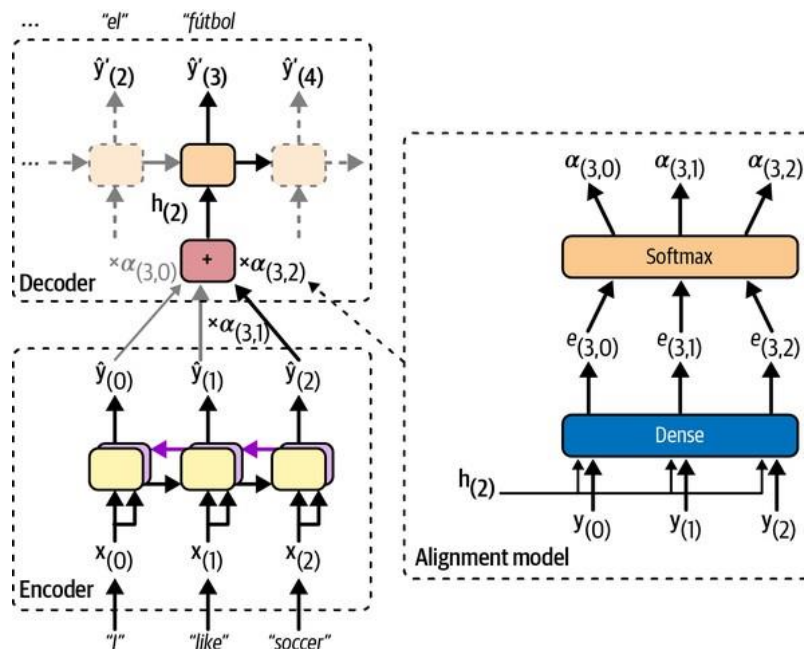
Với tất cả những điều này, bạn có thể nhận được các bản dịch khá tốt cho các câu tương đối ngắn. Thật không may, mô hình này sẽ rất tệ trong việc dịch các câu dài. Một lần nữa, vấn đề đến từ bộ nhớ ngắn hạn hạn chế của RNN. Cơ chế chú ý là sự đổi mới thay đổi cuộc chơi đã giải quyết vấn đề này.

Cơ chế chú ý (Attention Mechanisms)

Hãy xem xét đường dẫn từ từ “soccer” đến bản dịch “fútbol” trong Hình 16-3: nó khá dài! Điều này có nghĩa là một biểu diễn của từ này (cùng với tất cả các từ khác) cần phải được truyền qua nhiều bước trước khi nó thực sự được sử dụng. Chúng ta không thể làm cho đường dẫn này ngắn hơn sao?

Đây là ý tưởng cốt lõi trong một bài báo mang tính bước ngoặt năm 2014 của Dzmitry Bahdanau et al., nơi các tác giả đã giới thiệu một kỹ thuật cho phép bộ giải mã tập trung vào các từ thích hợp (được mã hóa bởi bộ mã hóa) ở mỗi bước thời gian. Ví dụ, tại bước thời gian mà bộ giải mã cần xuất ra từ “fútbol”, nó sẽ tập trung sự chú ý của mình vào từ “soccer”. Điều này có nghĩa là đường dẫn từ một từ đầu vào đến bản dịch của nó giờ đây ngắn hơn nhiều, vì vậy các hạn chế bộ nhớ ngắn hạn của RNN có ít tác động hơn. Cơ chế chú ý đã cách mạng hóa dịch máy thần kinh (và học sâu nói chung), cho phép cải thiện đáng kể trạng thái nghệ thuật, đặc biệt đối với các câu dài (ví dụ: hơn 30 từ).

Hình 16-7 hiển thị mô hình mã hóa-giải mã của chúng ta với cơ chế chú ý được thêm vào. Bên trái, bạn có bộ mã hóa và bộ giải mã. Thay vì chỉ gửi trạng thái ẩn cuối cùng của bộ mã hóa đến bộ giải mã, cũng như từ mục tiêu trước đó ở mỗi bước (vẫn được thực hiện, mặc dù không được hiển thị trong hình), bây giờ chúng ta cũng gửi tất cả các đầu ra của bộ mã hóa đến bộ giải mã. Vì bộ giải mã không thể xử lý tất cả các đầu ra bộ mã hóa này cùng một lúc, chúng cần được tổng hợp: ở mỗi bước thời gian, ô nhớ của bộ giải mã tính toán tổng trọng số của tất cả các đầu ra bộ mã hóa. Điều này xác định từ nào nó sẽ tập trung vào ở bước này. Trọng số $\alpha(t,i)$ là trọng số của đầu ra bộ mã hóa thứ i tại bước thời gian giải mã thứ t . Ví dụ, nếu trọng số $\alpha(3,2)$ lớn hơn nhiều so với trọng số $\alpha(3,0)$ và $\alpha(3,1)$, thì bộ giải mã sẽ chú ý nhiều hơn đến đầu ra của bộ mã hóa cho từ #2 (“soccer”) hơn là hai đầu ra khác, ít nhất là ở bước thời gian này. Phần còn lại của bộ giải mã hoạt động giống như trước: ở mỗi bước thời gian, ô nhớ nhận các đầu vào chúng ta vừa thảo luận, cộng với trạng thái ẩn từ bước thời gian trước đó, và cuối cùng (mặc dù không được biểu diễn trong sơ đồ) nó nhận từ mục tiêu từ bước thời gian trước đó (hoặc tại thời điểm suy luận, đầu ra từ bước thời gian trước đó).



Hình 16-7. Dịch máy thần kinh sử dụng mạng mã hóa-giải mã với mô hình chú ý

Nhưng các trọng số $\alpha(t,i)$ này đến từ đâu? Chà, chúng được tạo ra bởi một mạng nơ-ron nhỏ gọi là mô hình căn chỉnh (hoặc một lớp chú ý), được huấn luyện cùng với phần còn lại của mô hình mã hóa-giải mã. Mô hình căn chỉnh này được minh họa ở phía bên phải của Hình 16-7. Nó bắt đầu bằng một lớp Dense bao gồm một nơ-ron duy nhất xử lý từng đầu ra của bộ mã hóa, cùng với trạng thái ẩn trước đó của bộ giải mã (ví dụ: $h(2)$). Lớp này xuất ra một điểm (hoặc năng lượng) cho mỗi đầu ra bộ mã hóa (ví dụ: $e(3, 2)$): điểm này đo lường mức độ phù hợp của mỗi đầu ra với trạng thái ẩn trước đó của bộ giải mã. Ví dụ, trong Hình 16-7, mô hình đã xuất ra “me gusta el” (nghĩa là “Tôi thích”), vì vậy bây giờ nó đang mong đợi một danh từ: từ “soccer” là từ phù hợp nhất với trạng thái hiện tại, vì vậy nó nhận được điểm cao. Cuối cùng, tất cả các điểm đi qua một lớp softmax để nhận được trọng số cuối cùng cho mỗi đầu ra bộ mã hóa (ví dụ: $\alpha(3,2)$).

Tất cả các trọng số cho một bước thời gian giải mã nhất định cộng lại bằng 1. Cơ chế chú ý đặc biệt này được gọi là Bahdanau attention (đặt theo tên tác giả đầu tiên của bài báo năm 2014). Vì nó nối đầu ra bộ mã hóa với trạng thái ẩn trước đó của bộ giải mã, đôi khi nó được gọi là concatenative attention (hoặc additive attention).

Một cơ chế chú ý phổ biến khác, được gọi là Luong attention hoặc multiplicative attention, được đề xuất ngay sau đó, vào năm 2015, bởi Minh-Thang Luong et al. Vì mục tiêu của mô hình căn chỉnh là đo lường sự tương đồng giữa một trong các đầu ra của bộ mã hóa và trạng thái ẩn trước đó của bộ giải mã, các tác giả đã đề xuất chỉ đơn giản tính tích vô hướng (xem Chương 4) của hai vectơ này, vì đây thường là một thước đo tương đồng khá tốt, và phần cứng hiện đại có thể tính toán nó rất hiệu quả. Để điều này có thể thực hiện được, cả hai vectơ phải có cùng số chiều. Tích vô hướng cho ra một điểm, và tất cả các điểm (tại một bước thời gian giải mã nhất định) đi qua một lớp softmax để cho ra các trọng số cuối cùng, giống như trong Bahdanau attention. Một sự đơn giản hóa khác mà Luong et al. đề xuất là sử dụng trạng

thái ẩn của bộ giải mã ở bước thời gian hiện tại thay vì ở bước thời gian trước đó (tức là $h(t)$ thay vì $h(t-1)$), sau đó sử dụng đầu ra của cơ chế chú ý (ký hiệu $\tilde{h}(t)$) trực tiếp để tính toán các dự đoán của bộ giải mã, thay vì sử dụng nó để tính toán trạng thái ẩn hiện tại của bộ giải mã. Các nhà nghiên cứu cũng đề xuất một biến thể của cơ chế tích vô hướng trong đó các đầu ra bộ mã hóa trước tiên đi qua một lớp kết nối đầy đủ (không có số hạng bias) trước khi các tích vô hướng được tính toán. Điều này được gọi là cách tiếp cận tích vô hướng “chung”. Các nhà nghiên cứu đã so sánh cả hai cách tiếp cận tích vô hướng với cơ chế chú ý nối (thêm một vectơ tham số chia tỷ lệ v), và họ quan sát thấy rằng các biến thể tích vô hướng hoạt động tốt hơn chú ý nối. Vì lý do này, chú ý nối ít được sử dụng hơn bây giờ. Các phương trình cho ba cơ chế chú ý này được tóm tắt trong Phương trình 16-1.

Phương trình 16-1. Cơ chế chú ý

$$\tilde{h}_{(t)} = \sum_i \alpha_{(t,i)} \mathbf{y}_{(i)} \text{ with } \alpha_{(t,i)} = \frac{\exp(e_{(t,i)})}{\sum_{i'} \exp(e_{(t,i')})} \text{ and } e_{(t,i)} = \begin{cases} \mathbf{h}_{(t)}^\top \mathbf{y}_{(i)} & \text{do} \\ \mathbf{h}_{(t)}^\top \mathbf{W} \mathbf{y}_{(i)} & \text{ge} \\ \mathbf{v}^\top \tanh(\mathbf{W} [\mathbf{h}_{(t)}; \mathbf{y}_{(i)}]) & \text{co} \end{cases}$$

Keras cung cấp một lớp `tf.keras.layers.Attention` cho Luong attention, và một lớp `AdditiveAttention` cho Bahdanau attention. Hãy thêm Luong attention vào mô hình mã hóa-giải mã của chúng ta. Vì chúng ta sẽ cần truyền tất cả các đầu ra của bộ mã hóa đến lớp Attention, trước tiên chúng ta cần đặt `return_sequences=True` khi tạo bộ mã hóa:

```
encoder = tf.keras.layers.Bidirectional(
    tf.keras.layers.LSTM(256, return_sequences=True, return_state=True))
```

Tiếp theo, chúng ta cần tạo lớp chú ý và truyền trạng thái của bộ giải mã và đầu ra của bộ mã hóa cho nó. Tuy nhiên, để truy cập trạng thái của bộ giải mã ở mỗi bước, chúng ta sẽ cần viết một ô nhớ tùy chỉnh. Để đơn giản, hãy sử dụng đầu ra của bộ giải mã thay vì trạng thái của nó: trên thực tế, điều này cũng hoạt động tốt, và dễ mã hóa hơn nhiều. Sau đó, chúng ta chỉ cần truyền trực tiếp đầu ra của lớp chú ý đến lớp đầu ra, như đề xuất trong bài báo Luong attention:

```
attention_layer = tf.keras.layers.Attention()
attention_outputs = attention_layer([decoder_outputs, encoder_outputs])
output_layer = tf.keras.layers.Dense(vocab_size, activation="softmax")
Y_proba = output_layer(attention_outputs)
```

Và thế là xong! Nếu bạn huấn luyện mô hình này, bạn sẽ thấy rằng nó hiện xử lý các câu dài hơn nhiều. Ví dụ:

```
>>> translate("I
like soccer and also going to the beach") 'me gusta el fútbol y
también ir a la playa'
```

Tóm lại, lớp chú ý cung cấp một cách để tập trung sự chú ý của mô hình vào một phần của đầu vào.

Nhưng có một cách khác để nghĩ về lớp này: nó hoạt động như một cơ chế truy xuất bộ nhớ có thể phân biệt được.

Ví dụ, giả sử bộ mã hóa đã phân tích câu đầu vào “I like soccer”, và nó đã hiểu rằng từ “I” là chủ ngữ và từ “like” là động từ, vì vậy nó đã mã hóa thông tin này trong đầu ra của nó cho các từ này. Bây giờ giả sử bộ giải mã đã dịch chủ ngữ, và nó nghĩ rằng nó nên dịch động từ tiếp theo. Để làm điều này, nó cần tìm động từ từ câu đầu vào. Điều này tương tự như tra cứu từ điển: giống như bộ mã hóa đã tạo một từ điển {“subject”: “They”, “verb”: “played”, ...} và bộ giải mã muốn tra cứu giá trị tương ứng với khóa “verb”.

Tuy nhiên, mô hình không có các token rời rạc để biểu diễn các khóa (như “subject” hoặc “verb”); thay vào đó, nó có các biểu diễn vector hóa của các khái niệm này mà nó đã học được trong quá trình huấn luyện, vì vậy truy vấn mà nó sẽ sử dụng để tra cứu sẽ không khớp hoàn hảo với bất kỳ khóa nào trong từ điển. Giải pháp là tính toán một thước đo tương đồng giữa truy vấn và mỗi khóa trong từ điển, sau đó sử dụng hàm softmax để chuyển đổi các điểm tương đồng này thành các trọng số có tổng bằng 1. Như chúng ta đã thấy trước đó, đó chính xác là những gì lớp chú ý thực hiện. Nếu khóa biểu diễn động từ là tương tự nhất với truy vấn, thì trọng số của khóa đó sẽ gần bằng 1.

Tiếp theo, lớp chú ý tính toán tổng trọng số của các giá trị tương ứng: nếu trọng số của khóa “verb” gần bằng 1, thì tổng trọng số sẽ rất gần với biểu diễn của từ “played”. Đây là lý do tại sao các lớp Keras Attention và AdditiveAttention đều mong đợi một danh sách làm đầu vào, chứa hai hoặc ba mục: các truy vấn, các khóa, và tùy chọn các giá trị. Nếu bạn không truyền bất kỳ giá trị nào, thì chúng sẽ tự động bằng các khóa. Vì vậy, nhìn lại ví dụ mã trước đó, đầu ra của bộ giải mã là các truy vấn, và đầu ra của bộ mã hóa là cả các khóa và các giá trị. Đối với mỗi đầu ra bộ giải mã (tức là mỗi truy vấn), lớp chú ý trả về tổng trọng số của các đầu ra bộ mã hóa (tức là các khóa/giá trị) tương tự nhất với đầu ra của bộ giải mã.

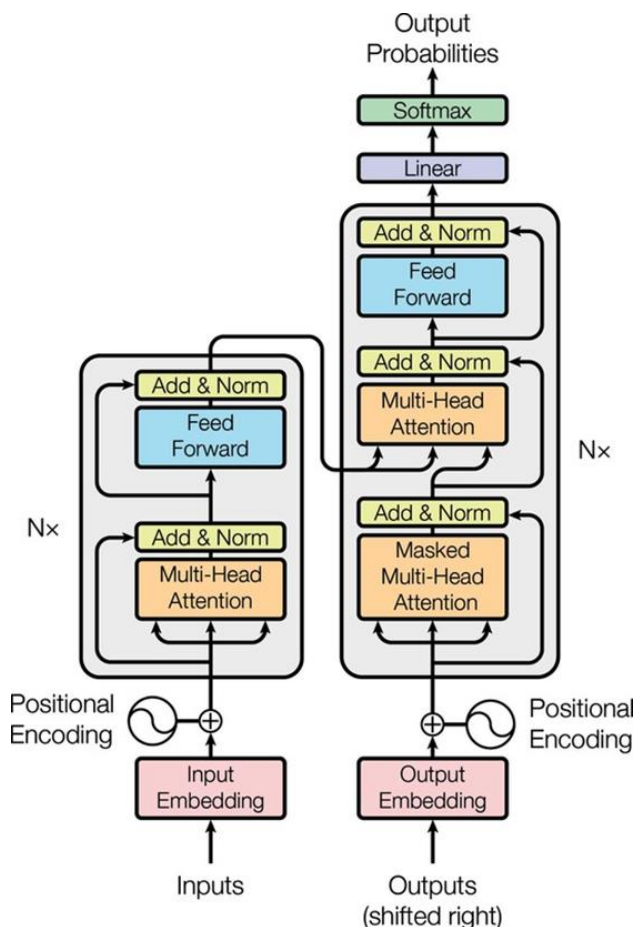
Điểm mấu chốt là cơ chế chú ý là một hệ thống truy xuất bộ nhớ có thể huấn luyện. Nó mạnh đến mức bạn thực sự có thể xây dựng các mô hình tiên tiến nhất chỉ bằng cách sử dụng cơ chế chú ý. Đi vào kiến trúc transformer.

Chủ đề “Attention Is All You Need: Kiến trúc Transformer gốc” nói về việc sử dụng cơ chế chú ý để cải thiện việc dịch máy thần kinh (NMT) mà không cần lớp hồi quy hoặc tích chập. Dưới đây là phần dịch của bạn:

Attention Is All You Need: Kiến trúc Transformer gốc

Trong một bài báo đột phá năm 2017, một nhóm các nhà nghiên cứu của Google đã gợi ý rằng “Attention Is All You Need”. Họ đã tạo ra một kiến trúc gọi là transformer, giúp cải thiện đáng kể trạng thái nghệ thuật trong NMT mà không sử dụng bất kỳ lớp hồi quy hoặc tích chập nào, chỉ sử dụng cơ chế chú ý (cộng với các lớp embedding, lớp dense, lớp chuẩn hóa và một vài thành phần khác). Vì mô hình không hồi quy, nó ít bị các vấn đề về đạo hàm biến mất hoặc bùng nổ như RNN, nó có thể được huấn luyện trong ít bước hơn, dễ dàng song song hóa trên nhiều GPU hơn và nó có thể nắm bắt các mẫu dài hạn tốt hơn RNN. Kiến trúc transformer gốc năm 2017 được thể hiện trong Hình 16-8.

Tóm lại, phần bên trái của Hình 16-8 là bộ mã hóa, và phần bên phải là bộ giải mã. Mỗi lớp embedding xuất ra một tensor 3D có hình dạng [kích thước batch, độ dài chuỗi, kích thước embedding]. Sau đó, các tensor dần dần được biến đổi khi chúng chảy qua transformer, nhưng hình dạng của chúng vẫn giữ nguyên.



Hình 16-8. Kiến trúc transformer gốc năm 2017

Nếu bạn sử dụng transformer cho NMT, thì trong quá trình huấn luyện, bạn phải cấp các câu tiếng Anh cho bộ mã hóa và các bản dịch tiếng Tây Ban Nha tương ứng cho bộ giải mã, với một token SOS bổ sung được chèn vào đầu mỗi câu. Tại thời điểm suy luận, bạn phải gọi transformer nhiều lần, tạo ra các bản dịch từng từ một và cấp các bản dịch một phần cho bộ giải mã ở mỗi vòng, giống như chúng ta đã làm trước đó trong hàm `translate()`.

Vai trò của bộ mã hóa là dần dần biến đổi các đầu vào — biểu diễn từ của câu tiếng Anh — cho đến khi biểu diễn của mỗi từ nắm bắt hoàn hảo ý nghĩa của từ đó, trong ngữ cảnh của câu. Ví dụ, nếu bạn cấp cho bộ mã hóa câu “I like soccer”, thì từ “like” sẽ bắt đầu với một biểu diễn khá mơ hồ, vì từ này có thể có nghĩa khác nhau trong các ngữ cảnh khác nhau: hãy nghĩ đến “I like soccer” so với “It’s like that”. Nhưng sau khi đi qua bộ mã hóa, biểu diễn của từ phải nắm bắt được ý nghĩa chính xác của “like” trong câu đã cho (tức là yêu thích), cũng như bất kỳ thông tin nào khác có thể cần thiết cho việc dịch (ví dụ: nó là một động từ).

Vai trò của bộ giải mã là dần dần biến đổi mỗi biểu diễn từ trong câu đã dịch thành một biểu diễn từ của từ tiếp theo trong bản dịch. Ví dụ, nếu câu cần dịch là “I like soccer”, và câu đầu vào của bộ giải mã là “<SOS> me gusta el fútbol”, thì sau khi đi qua bộ giải mã, biểu diễn từ của từ “el” sẽ được biến đổi thành một biểu diễn của từ “fútbol”. Tương tự, biểu diễn của từ “fútbol” sẽ được biến đổi thành một biểu diễn của token EOS.

Sau khi đi qua bộ giải mã, mỗi biểu diễn từ đi qua một lớp Dense cuối cùng với hàm kích hoạt softmax, hy vọng sẽ xuất ra xác suất cao cho từ tiếp theo đúng và xác suất thấp cho tất cả các từ khác. Câu được dự đoán phải là “me gusta el fútbol <EOS>”.

Đó là bức tranh tổng thể; bây giờ chúng ta hãy đi sâu vào Hình 16-8 một cách chi tiết hơn:

- Đầu tiên, lưu ý rằng cả bộ mã hóa và bộ giải mã đều chứa các mô-đun được xếp chồng N lần. Trong bài báo, $N = 6$. Các đầu ra cuối cùng của toàn bộ chồng bộ mã hóa được cấp cho bộ giải mã ở mỗi trong số N cấp độ này.
- Phóng to hơn, bạn có thể thấy rằng bạn đã quen thuộc với hầu hết các thành phần: có hai lớp embedding; một số kết nối bỏ qua (skip connection), mỗi kết nối theo sau bởi một lớp chuẩn hóa lớp; một số mô-đun truyền thẳng (feedforward) bao gồm hai lớp dense mỗi mô-đun (lớp đầu tiên sử dụng hàm kích hoạt ReLU, lớp thứ hai không có hàm kích hoạt); và cuối cùng lớp đầu ra là một lớp dense sử dụng hàm kích hoạt softmax. Bạn cũng có thể thêm một chút dropout sau các lớp chú ý và các mô-đun truyền thẳng, nếu cần. Vì tất cả các lớp này là phân phối thời gian (time-distributed), mỗi từ được xử lý độc lập với tất cả các từ khác. Nhưng làm thế nào chúng ta có thể dịch một câu bằng cách nhìn vào các từ hoàn toàn riêng biệt? Chà, chúng ta không thể, đó là lúc các thành phần mới xuất hiện:
 - Lớp chú ý đa đầu (multi-head attention) của bộ mã hóa cập nhật mỗi biểu diễn từ bằng cách chú ý đến (tức là tập trung vào) tất cả các từ khác trong cùng câu. Đó là nơi biểu diễn mơ hồ của từ “like” trở thành một biểu diễn phong phú hơn và chính xác hơn, nắm bắt ý nghĩa chính xác của nó trong câu đã cho. Chúng ta sẽ thảo luận chính xác cách hoạt động này ngay sau đây.
 - Lớp chú ý đa đầu được che (masked multi-head attention) của bộ giải mã làm điều tương tự, nhưng khi nó xử lý một từ, nó không chú ý đến các từ nằm sau nó: đó là một lớp nhân quả (causal layer). Ví dụ, khi nó xử lý từ “gusta”, nó chỉ chú ý đến các từ “<SOS> me gusta”, và nó bỏ qua các từ “el fútbol” (nếu không đó sẽ là gian lận).
 - Lớp chú ý đa đầu phía trên của bộ giải mã là nơi bộ giải mã chú ý đến các từ trong câu tiếng Anh. Đây được gọi là cross-attention, không phải self-attention trong trường hợp này. Ví dụ, bộ giải mã có thể sẽ chú ý kỹ đến từ “soccer” khi nó xử lý từ “el” và biến đổi biểu diễn của nó thành một biểu diễn của từ “fútbol”.
 - Các mã hóa vị trí (positional encodings) là các vector dày đặc (rất giống với embedding từ) đại diện cho vị trí của mỗi từ trong câu. Mã hóa vị trí thứ n được thêm vào embedding từ của từ thứ n trong mỗi câu. Điều này là cần thiết vì tất cả các lớp trong kiến trúc transformer bỏ qua vị trí từ: nếu không có mã hóa vị

trí, bạn có thể xáo trộn các chuỗi đầu vào, và nó sẽ chỉ xáo trộn các chuỗi đầu ra theo cùng một cách. Rõ ràng, thứ tự các từ rất quan trọng, đó là lý do tại sao chúng ta cần cung cấp thông tin vị trí cho transformer bằng cách nào đó: thêm mã hóa vị trí vào biểu diễn từ là một cách tốt để đạt được điều này.

Hãy đi sâu vào các thành phần mới của kiến trúc transformer một cách chi tiết hơn, bắt đầu với các mã hóa vị trí.

Mã hóa vị trí

Mã hóa vị trí là một vector dày đặc mã hóa vị trí của một từ trong một câu: mã hóa vị trí thứ i được thêm vào embedding từ của từ thứ i trong câu. Cách dễ nhất để triển khai điều này là sử dụng lớp Embedding và làm cho nó mã hóa tất cả các vị trí từ 0 đến độ dài chuỗi tối đa trong batch, sau đó thêm kết quả vào các embedding từ. Các quy tắc quảng bá (broadcasting) sẽ đảm bảo rằng các mã hóa vị trí được áp dụng cho mọi chuỗi đầu vào. Ví dụ, đây là cách thêm mã hóa vị trí vào đầu vào của bộ mã hóa và bộ giải mã:

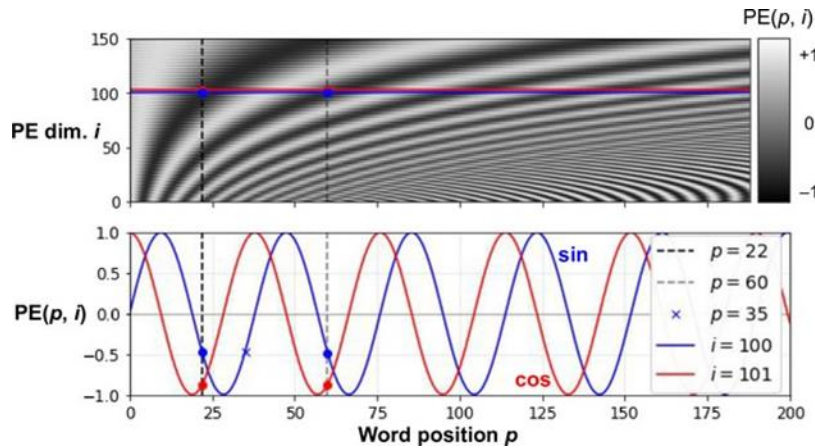
```
max_length = 50 # độ dài tối đa trong toàn bộ tập huấn luyện
embed_size = 128
pos_embed_layer = tf.keras.layers.Embedding(max_length, embed_size)
batch_max_len_enc = tf.shape(encoder_embeddings)[1]
encoder_in = encoder_embeddings + pos_embed_layer(tf.range(batch_max_len_enc)
)
batch_max_len_dec = tf.shape(decoder_embeddings)[1]
decoder_in = decoder_embeddings + pos_embed_layer(tf.range(batch_max_len_dec)
)
```

Lưu ý rằng việc triển khai này giả định rằng các embedding được biểu diễn dưới dạng các tensor thông thường, không phải các tensor lớn chằm. Bộ mã hóa và bộ giải mã chia sẻ cùng một lớp Embedding cho các mã hóa vị trí, vì chúng có cùng kích thước embedding (đây thường là trường hợp).

Thay vì sử dụng các mã hóa vị trí có thể huấn luyện, các tác giả của bài báo transformer đã chọn sử dụng các mã hóa vị trí cố định, dựa trên các hàm sin và cosin ở các tần số khác nhau. Ma trận mã hóa vị trí P được định nghĩa trong Phương trình 16-2 và được biểu diễn ở trên cùng của Hình 16-9 (chuyển vị), trong đó $P_{p,i}$ là thành phần thứ i của mã hóa cho từ nằm ở vị trí thứ p trong câu.

Phương trình 16-2. Mã hóa vị trí sin/cosin

$$P_{p,i} = \begin{cases} \sin\left(p/10000^{i/d}\right) & \text{if } i \text{ is even} \\ \cos\left(p/10000^{(i-1)/d}\right) & \text{if } i \text{ is odd} \end{cases}$$



Hình 16-9. Ma trận mã hóa vị trí sin/cosin (chuyển vị, trên cùng) với tiêu điểm vào hai giá trị của i (dưới cùng)

Giải pháp này có thể cho cùng hiệu suất như các mã hóa vị trí có thể huấn luyện, và nó có thể mở rộng đến các câu dài tùy ý mà không cần thêm bất kỳ tham số nào vào mô hình (tuy nhiên, khi có một lượng lớn dữ liệu tiền huấn luyện, các mã hóa vị trí có thể huấn luyện thường được ưu tiên). Sau khi các mã hóa vị trí này được thêm vào các embedding từ, phần còn lại của mô hình có quyền truy cập vào vị trí tuyệt đối của mỗi từ trong câu vì có một mã hóa vị trí duy nhất cho mỗi vị trí (ví dụ: mã hóa vị trí cho từ nằm ở vị trí thứ 22 trong một câu được biểu diễn bằng đường nét đứt dọc ở phía trên bên trái của Hình 16-9, và bạn có thể thấy rằng nó là duy nhất đối với vị trí đó). Hơn nữa, việc lựa chọn các hàm dao động (sin và cosin) giúp mô hình có thể học cả các vị trí tương đối. Ví dụ, các từ cách nhau 38 từ (ví dụ: ở vị trí $p = 22$ và $p = 60$) luôn có cùng giá trị mã hóa vị trí trong các chiều mã hóa $i = 100$ và $i = 101$, như bạn có thể thấy trong Hình 16-9. Điều này giải thích tại sao chúng ta cần cả sin và cosin cho mỗi tần số: nếu chúng ta chỉ sử dụng sin (sóng xanh ở $i = 100$), mô hình sẽ không thể phân biệt các vị trí $p = 22$ và $p = 35$ (được đánh dấu bằng dấu thập).

Không có lớp `PositionalEncoding` trong TensorFlow, nhưng không quá khó để tạo một lớp. Vì lý do hiệu quả, chúng ta tính toán trước ma trận mã hóa vị trí trong hàm tạo. Phương thức `call()` chỉ cắt ma trận mã hóa này theo độ dài tối đa của chuỗi đầu vào, và nó thêm chúng vào các đầu vào. Chúng ta cũng đặt `supports_masking=True` để lan truyền mặt nạ tự động của đầu vào đến lớp tiếp theo:

```
import numpy as np

class PositionalEncoding(tf.keras.layers.Layer):
    def __init__(self, max_length, embed_size, dtype=tf.float32, **kwargs):
        super().__init__(dtype=dtype, **kwargs)
        assert embed_size % 2 == 0, "embed_size must be even"
        p, i = np.meshgrid(np.arange(max_length),
                            2 * np.arange(embed_size // 2))
        pos_emb = np.empty((1, max_length, embed_size))
        pos_emb[0, :, ::2] = np.sin(p / 10_000 ** (i / embed_size)).T
        pos_emb[0, :, 1::2] = np.cos(p / 10_000 ** (i / embed_size)).T
```



```

self.pos_encodings = tf.constant(pos_emb.astype(self.dtype))
self.supports_masking = True

def call(self, inputs):
    batch_max_length = tf.shape(inputs)[1]
    return inputs + self.pos_encodings[:, :batch_max_length]

```

Hãy sử dụng lớp này để thêm mã hóa vị trí vào đầu vào của bộ mã hóa:

```

pos_embed_layer = PositionalEncoding(max_length, embed_size)
encoder_in = pos_embed_layer(encoder_embeddings)
decoder_in = pos_embed_layer(decoder_embeddings)

```

Bây giờ chúng ta hãy xem sâu hơn vào trái tim của mô hình transformer, tại lớp chú ý đa đầu.

Chú ý đa đầu (Multi-head attention)

Để hiểu cách hoạt động của lớp chú ý đa đầu, trước tiên chúng ta phải hiểu lớp chú ý tích vô hướng có tỷ lệ (scaled dot-product attention), mà nó dựa vào. Phương trình của nó được thể hiện trong Phương trình 16-3, dưới dạng vector hóa. Nó giống như Luong attention, ngoại trừ một hệ số tỷ lệ.

Phương trình 16-3. Chú ý tích vô hướng có tỷ lệ

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax} \left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_{\text{keys}}}} \right) \mathbf{V}$$

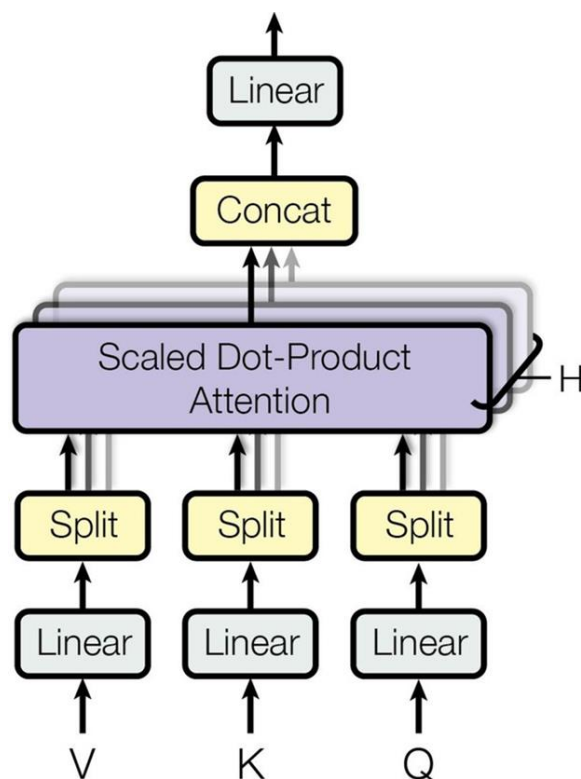
Trong phương trình này:

- $\mathbf{Q}$ là ma trận chứa một hàng cho mỗi truy vấn. Hình dạng của nó là $[\text{nqueries}, d_{\text{keys}}]$, trong đó nqueries là số lượng truy vấn và d_{keys} là số chiều của mỗi truy vấn và mỗi khóa.
- $\mathbf{K}$ là ma trận chứa một hàng cho mỗi khóa. Hình dạng của nó là $[\text{nkeys}, d_{\text{keys}}]$, trong đó nkeys là số lượng khóa và giá trị.
- $\mathbf{V}$ là ma trận chứa một hàng cho mỗi giá trị. Hình dạng của nó là $[\text{nkeys}, d_{\text{values}}]$, trong đó d_{values} là số chiều của mỗi giá trị.
- Hình dạng của $\mathbf{Q}\mathbf{K}^T$ là $[\text{nqueries}, \text{nkeys}]$: nó chứa một điểm tương đồng cho mỗi cặp truy vấn/ khóa. Để ngăn ma trận này trở nên quá lớn, chuỗi đầu vào không được quá dài (chúng ta sẽ thảo luận cách khắc phục hạn chế này sau trong chương này). Đầu ra của hàm softmax có cùng hình dạng, nhưng tất cả các hàng tổng cộng bằng 1.
- Đầu ra cuối cùng có hình dạng $[\text{nqueries}, d_{\text{values}}]$: có một hàng cho mỗi truy vấn, trong đó mỗi hàng biểu diễn kết quả truy vấn (tổng trọng số của các giá trị).
- Hệ số tỷ lệ $1/\sqrt{d_{\text{keys}}}$ giảm tỷ lệ các điểm tương đồng để tránh làm bão hòa hàm softmax, điều này sẽ dẫn đến các gradient rất nhỏ.

- Có thể che một số cặp khóa/giá trị bằng cách thêm một giá trị âm rất lớn vào các điểm tương đồng tương ứng, ngay trước khi tính toán softmax. Điều này hữu ích trong lớp chú ý đa đầu được che.
- Nếu bạn đặt `use_scale=True` khi tạo một lớp `tf.keras.layers.Attention`, thì nó sẽ tạo ra một tham số bổ sung cho phép lớp học cách giảm tỷ lệ các điểm tương đồng một cách hợp lý. Chú ý tích vô hướng có tỷ lệ được sử dụng trong mô hình transformer gần như giống hệt, ngoại trừ việc nó luôn giảm tỷ lệ các điểm tương đồng bằng cùng một hệ số, $1/(\sqrt{d_{keys}})$.

Lưu ý rằng đầu vào của lớp Attention giống như Q, K và V, ngoại trừ một chiều batch bổ sung (chiều đầu tiên). Bên trong, lớp tính toán tất cả các điểm chú ý cho tất cả các câu trong batch chỉ bằng một lần gọi `tf.matmul(queries, keys)`: điều này làm cho nó cực kỳ hiệu quả. Thật vậy, trong TensorFlow, nếu A và B là các tensor có nhiều hơn hai chiều — ví dụ, có hình dạng [2, 3, 4, 5] và [2, 3, 5, 6] tương ứng — thì `tf.matmul(A, B)` sẽ xử lý các tensor này như các mảng 2×3 trong đó mỗi ô chứa một ma trận, và nó sẽ nhân các ma trận tương ứng: ma trận ở hàng thứ i và cột thứ j trong A sẽ được nhân với ma trận ở hàng thứ i và cột thứ j trong B. Vì tích của một ma trận 4×5 với một ma trận 5×6 là một ma trận 4×6 , `tf.matmul(A, B)` sẽ trả về một mảng có hình dạng [2, 3, 4, 6].

Bây giờ chúng ta đã sẵn sàng xem xét lớp chú ý đa đầu. Kiến trúc của nó được thể hiện trong Hình 16-10.



Hình 16-10. Kiến trúc lớp chú ý đa đầu

Như bạn có thể thấy, nó chỉ là một tập hợp các lớp chú ý tích vô hướng có tỷ lệ, mỗi lớp được đặt trước bởi một phép biến đổi tuyến tính của các giá trị, khóa và truy vấn (tức là một lớp dense phân phối thời gian không có hàm kích hoạt). Tất cả các đầu ra chỉ đơn giản được nối lại, và chúng đi qua một phép biến đổi tuyến tính cuối cùng (một lần nữa, phân phối thời gian).

Nhưng tại sao? Trực giác đằng sau kiến trúc này là gì? Chà, hãy xem xét lại từ “like” trong câu “I like soccer”. Bộ mã hóa đủ thông minh để mã hóa thực tế rằng nó là một động từ. Nhưng biểu diễn từ cũng bao gồm vị trí của nó trong văn bản, nhờ các mã hóa vị trí, và nó có lẽ bao gồm nhiều đặc điểm khác hữu ích cho bản dịch của nó, chẳng hạn như thực tế là nó ở thì hiện tại đơn. Tóm lại, biểu diễn từ mã hóa nhiều đặc điểm khác nhau của từ. Nếu chúng ta chỉ sử dụng một lớp chú ý tích vô hướng có tỷ lệ duy nhất, chúng ta sẽ chỉ có thể truy vấn tất cả các đặc điểm này trong một lần.

Đây là lý do tại sao lớp chú ý đa đầu áp dụng nhiều phép biến đổi tuyến tính khác nhau của các giá trị, khóa và truy vấn: điều này cho phép mô hình áp dụng nhiều phép chiếu khác nhau của biểu diễn từ vào các không gian con khác nhau, mỗi không gian con tập trung vào một tập hợp con các đặc điểm của từ. Có lẽ một trong các lớp tuyến tính sẽ chiếu biểu diễn từ vào một không gian con nơi tất cả những gì còn lại là thông tin từ đó là một động từ, một lớp tuyến tính khác sẽ chỉ trích xuất thực tế rằng nó ở thì hiện tại đơn, v.v. Sau đó, các lớp chú ý tích vô hướng có tỷ lệ triển khai giai đoạn tra cứu, và cuối cùng chúng ta nối tất cả các kết quả và chiếu chúng trở lại không gian ban đầu.

Keras bao gồm một lớp `tf.keras.layers.MultiHeadAttention`, vì vậy bây giờ chúng ta có mọi thứ cần thiết để xây dựng phần còn lại của transformer. Hãy bắt đầu với bộ mã hóa hoàn chỉnh, giống hệt như trong Hình 16-8, ngoại trừ chúng ta sử dụng một chồng hai khối ($N = 2$) thay vì sáu, vì chúng ta không có một tập huấn luyện lớn, và chúng ta cũng thêm một chút dropout:

```
N = 2 # thay vì 6
num_heads = 8
dropout_rate = 0.1
n_units = 128 # cho lớp dense đầu tiên trong mỗi khối feedforward
encoder_pad_mask = tf.math.not_equal(encoder_input_ids, 0)[: , tf.newaxis]
Z = encoder_in

for _ in range(N):
    skip = Z
    attn_layer = tf.keras.layers.MultiHeadAttention(
        num_heads=num_heads, key_dim=embed_size, dropout=dropout_rate)
    Z = attn_layer(Z, value=Z, attention_mask=encoder_pad_mask)
    Z = tf.keras.layers.LayerNormalization()(tf.keras.layers.Add()([Z, skip])
    )
    skip = Z
    Z = tf.keras.layers.Dense(n_units, activation="relu")(Z)
    Z = tf.keras.layers.Dense(embed_size)(Z)
    Z = tf.keras.layers.Dropout(dropout_rate)(Z)
```

```
Z = tf.keras.layers.LayerNormalization()(tf.keras.layers.Add()([Z, skip])
)
```

Đoạn mã này hầu hết là dễ hiểu, ngoại trừ một điều: che mặt (masking). Tại thời điểm viết bài, lớp MultiHeadAttention không hỗ trợ che mặt tự động, vì vậy chúng ta phải xử lý thủ công. Làm thế nào chúng ta có thể làm điều đó?

Lớp MultiHeadAttention chấp nhận một đối số attention_mask, là một tensor Boolean có hình dạng [kích thước batch, độ dài truy vấn tối đa, độ dài giá trị tối đa]: đối với mỗi token trong mỗi chuỗi truy vấn, mặt nạ này cho biết token nào trong chuỗi giá trị tương ứng nên được chú ý. Chúng ta muốn nói với lớp MultiHeadAttention bỏ qua tất cả các token đệm trong các giá trị. Vì vậy, trước tiên, chúng ta tính toán mặt nạ đệm bằng cách sử dụng `tf.math.not_equal(encoder_input_ids, 0)`. Điều này trả về một tensor Boolean có hình dạng [kích thước batch, độ dài chuỗi tối đa]. Sau đó, chúng ta chèn một trục thứ hai bằng cách sử dụng `[:, tf.newaxis]`, để nhận được một mặt nạ có hình dạng [kích thước batch, 1, độ dài chuỗi tối đa]. Điều này cho phép chúng ta sử dụng mặt nạ này làm attention_mask khi gọi lớp MultiHeadAttention: nhờ tính năng broadcasting, cùng một mặt nạ sẽ được sử dụng cho tất cả các token trong mỗi truy vấn. Bằng cách này, các token đệm trong các giá trị sẽ được bỏ qua một cách chính xác.

Tuy nhiên, lớp sẽ tính toán đầu ra cho mọi token truy vấn, bao gồm cả các token đệm. Chúng ta cần che đầu ra tương ứng với các token đệm này. Nhớ lại rằng chúng ta đã sử dụng mask_zero trong các lớp Embedding, và chúng ta đã đặt supports_masking thành True trong lớp PositionalEncoding, vì vậy mặt nạ tự động đã được lan truyền đến đầu vào của lớp MultiHeadAttention (encoder_in). Chúng ta có thể tận dụng điều này trong kết nối bỏ qua: thực tế, lớp Add hỗ trợ che mặt tự động, vì vậy khi chúng ta thêm Z và skip (ban đầu bằng encoder_in), đầu ra sẽ tự động được che chính xác. Chà! Che mặt đòi hỏi nhiều giải thích hơn là mã.

Bây giờ đến bộ giải mã! Một lần nữa, che mặt sẽ là phần khó khăn duy nhất, vì vậy hãy bắt đầu với điều đó. Lớp chú ý đa đầu đầu tiên là một lớp tự chú ý (self-attention layer), giống như trong bộ mã hóa, nhưng nó là một lớp chú ý đa đầu được che, nghĩa là nó có tính nhân quả: nó phải bỏ qua tất cả các token trong tương lai. Vì vậy, chúng ta cần hai mặt nạ: một mặt nạ đệm và một mặt nạ nhân quả. Hãy tạo chúng:

```
decoder_pad_mask = tf.math.not_equal(decoder_input_ids, 0)[:, tf.newaxis]
causal_mask = tf.linalg.band_part( # tạo một ma trận tam giác dưới
    tf.ones((batch_max_len_dec, batch_max_len_dec), tf.bool), -1, 0)
```

Mặt nạ đệm chính xác giống như mặt nạ chúng ta đã tạo cho bộ mã hóa, ngoại trừ nó dựa trên đầu vào của bộ giải mã thay vì của bộ mã hóa. Mặt nạ nhân quả được tạo bằng hàm `tf.linalg.band_part()`, hàm này nhận một tensor và trả về một bản sao với tất cả các giá trị bên ngoài một dải chéo được đặt thành 0. Với các đối số này, chúng ta nhận được một ma trận vuông có kích thước batch_max_len_dec (độ dài tối đa của chuỗi đầu vào trong batch), với các số 1 ở tam giác dưới bên trái và các số 0 ở phía trên bên phải. Nếu chúng ta sử dụng mặt nạ này làm mặt nạ chú ý, chúng ta sẽ nhận được chính xác những gì chúng ta muốn: token truy vấn đầu tiên sẽ chỉ chú ý đến token giá trị đầu tiên, token thứ hai sẽ chỉ chú ý đến

hai token đầu tiên, token thứ ba sẽ chỉ chú ý đến ba token đầu tiên, v.v. Nói cách khác, các token truy vấn không thể chú ý đến bất kỳ token giá trị nào trong tương lai.

Bây giờ hãy xây dựng bộ giải mã:

```
encoder_outputs = Z # hãy lưu các đầu ra cuối cùng của bộ mã hóa
Z = decoder_in # bộ giải mã bắt đầu với đầu vào riêng của nó

for _ in range(N):
    skip = Z
    attn_layer = tf.keras.layers.MultiHeadAttention(
        num_heads=num_heads, key_dim=embed_size, dropout=dropout_rate)
    Z = attn_layer(Z, value=Z, attention_mask=causal_mask & decoder_pad_mask)
    Z = tf.keras.layers.LayerNormalization()(tf.keras.layers.Add()([Z, skip])
    )
    skip = Z
    attn_layer = tf.keras.layers.MultiHeadAttention(
        num_heads=num_heads, key_dim=embed_size, dropout=dropout_rate)
    Z = attn_layer(Z, value=encoder_outputs, attention_mask=encoder_pad_mask)
    Z = tf.keras.layers.LayerNormalization()(tf.keras.layers.Add()([Z, skip])
    )
    skip = Z
    Z = tf.keras.layers.Dense(n_units, activation="relu")(Z)
    Z = tf.keras.layers.Dense(embed_size)(Z)
    Z = tf.keras.layers.LayerNormalization()(tf.keras.layers.Add()([Z, skip])
    )
```

Đối với lớp chú ý đầu tiên, chúng ta sử dụng `causal_mask` & `decoder_pad_mask` để che cả token đệm và token tương lai. Mặt nạ nhân quả chỉ có hai chiều: nó thiếu chiều batch, nhưng không sao vì broadcasting đảm bảo rằng nó được sao chép trên tất cả các instance trong batch.

Đối với lớp chú ý thứ hai, không có gì đặc biệt. Điều duy nhất cần lưu ý là chúng ta đang sử dụng `encoder_pad_mask`, không phải `decoder_pad_mask`, vì lớp chú ý này sử dụng đầu ra cuối cùng của bộ mã hóa làm giá trị của nó.

Chúng ta gần xong rồi. Chúng ta chỉ cần thêm lớp đầu ra cuối cùng, tạo mô hình, biên dịch nó và huấn luyện nó:

```
Y_proba = tf.keras.layers.Dense(vocab_size, activation="softmax")(Z)
model = tf.keras.Model(inputs=[encoder_inputs, decoder_inputs],
                        outputs=[Y_proba])
model.compile(loss="sparse_categorical_crossentropy", optimizer="nadam",
              metrics=["accuracy"])
model.fit((X_train, X_train_dec), Y_train, epochs=10,
        validation_data=((X_valid, X_valid_dec), Y_valid))
```

Chúc mừng! Bạn đã xây dựng một transformer hoàn chỉnh từ đầu, và huấn luyện nó để dịch tự động. Điều này đang trở nên khá nâng cao!

Nhưng lĩnh vực này không dừng lại ở đó. Bây giờ chúng ta hãy khám phá một số tiến bộ gần đây.

Tuyệt vời các mô hình Transformer

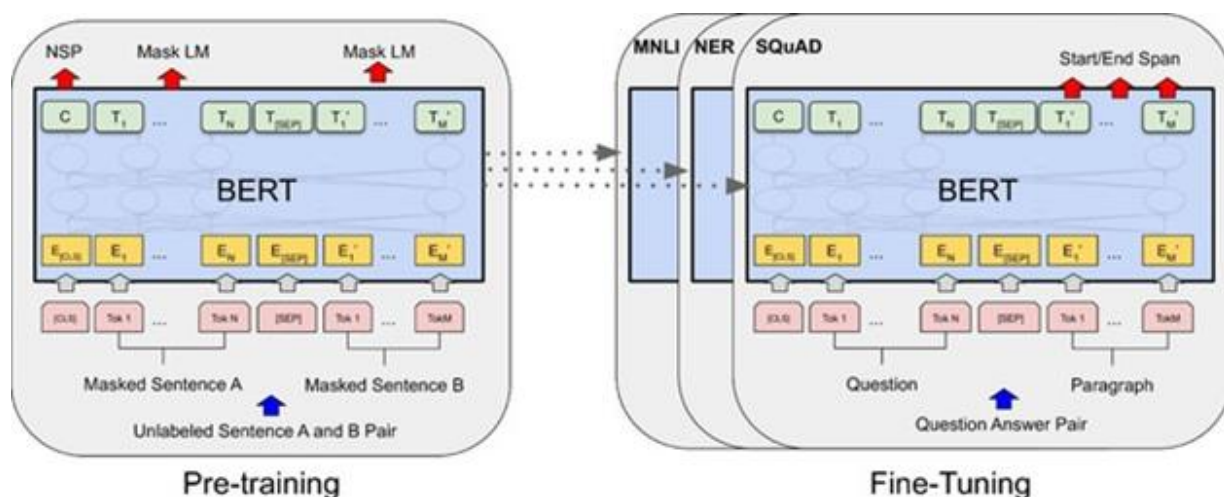
Năm 2018 được gọi là “thời điểm ImageNet cho NLP”. Kể từ đó, sự tiến bộ thật đáng kinh ngạc, với các kiến trúc dựa trên transformer ngày càng lớn hơn được huấn luyện trên các tập dữ liệu khổng lồ.

Đầu tiên, bài báo GPT của Alec Radford và các nhà nghiên cứu khác của OpenAI một lần nữa chứng minh hiệu quả của việc tiền huấn luyện không giám sát, giống như các bài báo ELMo và ULMFiT trước đó, nhưng lần này sử dụng kiến trúc giống transformer. Các tác giả đã tiền huấn luyện một kiến trúc lớn nhưng khá đơn giản bao gồm một chồng 12 mô-đun transformer chỉ sử dụng các lớp chú ý đa đầu được che, giống như trong bộ giải mã transformer gốc. Họ đã huấn luyện nó trên một tập dữ liệu rất lớn, sử dụng cùng kỹ thuật tự hồi quy mà chúng ta đã sử dụng cho char-RNN kiểu Shakespeare của mình: chỉ cần dự đoán token tiếp theo. Đây là một dạng học tự giám sát. Sau đó, họ đã tinh chỉnh nó trên các tác vụ ngôn ngữ khác nhau, chỉ sử dụng các điều chỉnh nhỏ cho mỗi tác vụ. Các tác vụ khá đa dạng: chúng bao gồm phân loại văn bản, suy luận (liệu câu A có áp đặt, liên quan hoặc ngụ ý câu B như một hệ quả tất yếu hay không), tương đồng (ví dụ: “Nice weather today” rất giống với “It is sunny”), và trả lời câu hỏi (cho một vài đoạn văn bản cung cấp ngữ cảnh, mô hình phải trả lời một số câu hỏi trắc nghiệm).

Sau đó, bài báo BERT của Google đã ra đời: nó cũng chứng minh hiệu quả của việc tiền huấn luyện tự giám sát trên một corpus lớn, sử dụng một kiến trúc tương tự GPT nhưng chỉ với các lớp chú ý đa đầu không che, giống như trong bộ mã hóa transformer gốc. Điều này có nghĩa là mô hình tự nhiên là hai chiều; do đó chữ B trong BERT (Bidirectional Encoder Representations from Transformers - Biểu diễn mã hóa hai chiều từ Transformers). Quan trọng nhất, các tác giả đã đề xuất hai tác vụ tiền huấn luyện giải thích hầu hết sức mạnh của mô hình:

- **Mô hình ngôn ngữ được che (MLM)** Mỗi từ trong một câu có 15% xác suất bị che, và mô hình được huấn luyện để dự đoán các từ bị che. Ví dụ, nếu câu gốc là “She had fun at the birthday party”, thì mô hình có thể được cung cấp câu “She <mask> fun at the <mask> party” và nó phải dự đoán các từ “had” và “birthday” (các đầu ra khác sẽ bị bỏ qua). Để chính xác hơn, mỗi từ được chọn có 80% cơ hội bị che, 10% cơ hội được thay thế bằng một từ ngẫu nhiên (để giảm sự khác biệt giữa tiền huấn luyện và tinh chỉnh, vì mô hình sẽ không thấy các token <mask> trong quá trình tinh chỉnh), và 10% cơ hội được giữ nguyên (để làm lệch mô hình về câu trả lời đúng).
- **Dự đoán câu tiếp theo (NSP)** Mô hình được huấn luyện để dự đoán liệu hai câu có liên tiếp nhau hay không. Ví dụ, nó nên dự đoán rằng “The dog sleeps” và “It snores loudly” là các câu liên tiếp, trong khi “The dog sleeps” và “The Earth orbits the Sun” không liên tiếp. Nghiên cứu sau này cho thấy rằng NSP không quan trọng như ban đầu nghĩ, vì vậy nó đã bị loại bỏ trong hầu hết các kiến trúc sau này.

Mô hình được huấn luyện trên hai tác vụ này đồng thời (xem Hình 16-11). Đối với tác vụ NSP, các tác giả đã chèn một token lớp (<CLS>) vào đầu mỗi đầu vào, và token đầu ra tương ứng đại diện cho dự đoán của mô hình: câu B theo sau câu A, hoặc không. Hai câu đầu vào được nối lại, chỉ được phân tách bằng một token phân tách đặc biệt (<SEP>), và chúng được cấp làm đầu vào cho mô hình. Để giúp mô hình biết mỗi token đầu vào thuộc về câu nào, một embedding phân đoạn được thêm vào trên mỗi embedding vị trí của token: chỉ có hai embedding phân đoạn có thể có, một cho câu A và một cho câu B. Đối với tác vụ MLM, một số từ đầu vào bị che (như chúng ta vừa thấy) và mô hình cố gắng dự đoán những từ đó là gì. Hàm mất mát chỉ được tính toán trên dự đoán NSP và các token bị che, không phải trên các token không che.



Hình 16-11. Quy trình huấn luyện và tinh chỉnh BERT

Sau giai đoạn tiền huấn luyện không giám sát này trên một corpus văn bản rất lớn, mô hình sau đó được tinh chỉnh trên nhiều tác vụ khác nhau, thay đổi rất ít cho mỗi tác vụ. Ví dụ, đối với phân loại văn bản như phân tích cảm xúc, tất cả các token đầu ra đều bị bỏ qua ngoại trừ token đầu tiên, tương ứng với token lớp, và một lớp đầu ra mới thay thế lớp trước đó, vốn chỉ là một lớp phân loại nhị phân cho NSP.

Vào tháng 2 năm 2019, chỉ vài tháng sau khi BERT được xuất bản, Alec Radford, Jeffrey Wu và các nhà nghiên cứu khác của OpenAI đã xuất bản bài báo GPT-2, đề xuất một kiến trúc rất giống GPT, nhưng lớn hơn nữa (với hơn 1,5 tỷ tham số!). Các nhà nghiên cứu đã chỉ ra rằng mô hình GPT mới và cải tiến có thể thực hiện học zero-shot (ZSL), nghĩa là nó có thể đạt được hiệu suất tốt trên nhiều tác vụ mà không cần tinh chỉnh. Đây chỉ là sự khởi đầu của một cuộc đua hướng tới các mô hình ngày càng lớn hơn: Google's Switch Transformers (được giới thiệu vào tháng 1 năm 2021) đã sử dụng 1 nghìn tỷ tham số, và ngay sau đó các mô hình lớn hơn nhiều đã ra đời, chẳng hạn như mô hình Wu Dao 2.0 của Học viện Trí tuệ Nhân tạo Bắc Kinh (BAII), được công bố vào tháng 6 năm 2021.

Một hệ quả đáng tiếc của xu hướng hướng tới các mô hình khổng lồ này là chỉ các tổ chức được tài trợ tốt mới có đủ khả năng huấn luyện các mô hình như vậy: nó có thể dễ dàng tốn hàng trăm nghìn đô la hoặc hơn. Và năng lượng cần thiết để huấn luyện một mô hình duy

nhất tương ứng với mức tiêu thụ điện của một hộ gia đình Mỹ trong vài năm; nó hoàn toàn không thân thiện với môi trường. Nhiều mô hình trong số này quá lớn đến mức không thể sử dụng trên phần cứng thông thường: chúng sẽ không vừa RAM, và chúng sẽ chậm khủng khiếp. Cuối cùng, một số quá tốn kém đến mức chúng không được phát hành công khai.

May mắn thay, các nhà nghiên cứu tài tình đang tìm ra những cách mới để giảm kích thước transformer và làm cho chúng hiệu quả hơn về dữ liệu. Ví dụ, mô hình DistilBERT, được giới thiệu vào tháng 10 năm 2019 bởi Victor Sanh et al. từ Hugging Face, là một mô hình transformer nhỏ và nhanh dựa trên BERT. Nó có sẵn trên trung tâm mô hình xuất sắc của Hugging Face, cùng với hàng nghìn mô hình khác — bạn sẽ thấy một ví dụ sau trong chương này. DistilBERT được huấn luyện bằng cách sử dụng chưng cất (do đó có tên): điều này có nghĩa là truyền kiến thức từ mô hình giáo viên sang mô hình học sinh, thường nhỏ hơn nhiều so với mô hình giáo viên. Điều này thường được thực hiện bằng cách sử dụng các xác suất dự đoán của giáo viên cho mỗi instance huấn luyện làm mục tiêu cho học sinh. Đáng ngạc nhiên, chưng cất thường hoạt động tốt hơn là huấn luyện học sinh từ đầu trên cùng một tập dữ liệu với giáo viên! Thật vậy, học sinh được hưởng lợi từ các nhãn sắc thái hơn của giáo viên.

Nhiều kiến trúc transformer khác đã ra đời sau BERT, gần như hàng tháng, thường cải thiện trạng thái nghệ thuật trên tất cả các tác vụ NLP: XLNet (tháng 6 năm 2019), RoBERTa (tháng 7 năm 2019), StructBERT (tháng 8 năm 2019), ALBERT (tháng 9 năm 2019), T5 (tháng 10 năm 2019), ELECTRA (tháng 3 năm 2020), GPT-3 (tháng 5 năm 2020), DeBERTa (tháng 6 năm 2020), Switch Transformers (tháng 1 năm 2021), Wu Dao 2.0 (tháng 6 năm 2021), Gopher (tháng 12 năm 2021), GPT-NeoX-20B (tháng 2 năm 2022), Chinchilla (tháng 3 năm 2022), OPT (tháng 5 năm 2022), và danh sách cứ tiếp tục dài. Mỗi mô hình này đều mang đến những ý tưởng và kỹ thuật mới, nhưng tôi đặc biệt thích bài báo T5 của các nhà nghiên cứu Google: nó định hình tất cả các tác vụ NLP dưới dạng văn bản-thành-văn bản, sử dụng một transformer mã hóa-giải mã. Ví dụ, để dịch “I like soccer” sang tiếng Tây Ban Nha, bạn chỉ cần gọi mô hình với câu đầu vào “translate English to Spanish: I like soccer” và nó xuất ra “me gusta el fútbol”. Để tóm tắt một đoạn văn, bạn chỉ cần nhập “summarize:” theo sau đoạn văn, và nó xuất ra bản tóm tắt. Để phân loại, bạn chỉ cần thay đổi tiền tố thành “classify:” và mô hình xuất ra tên lớp, dưới dạng văn bản. Điều này đơn giản hóa việc sử dụng mô hình, và nó cũng giúp có thể tiền huấn luyện nó trên nhiều tác vụ hơn.

Cuối cùng nhưng không kém phần quan trọng, vào tháng 4 năm 2022, các nhà nghiên cứu Google đã sử dụng một nền tảng huấn luyện quy mô lớn mới có tên Pathways (chúng ta sẽ thảo luận ngắn gọn trong Chương 19) để huấn luyện một mô hình ngôn ngữ khổng lồ có tên Pathways Language Model (PaLM), với 540 tỷ tham số, sử dụng hơn 6.000 TPU.

Ngoài kích thước đáng kinh ngạc của nó, mô hình này là một transformer tiêu chuẩn, chỉ sử dụng bộ giải mã (tức là với các lớp chú ý đa đầu được che), với một vài điều chỉnh nhỏ (xem bài báo để biết chi tiết). Mô hình này đã đạt được hiệu suất đáng kinh ngạc trên tất cả các loại tác vụ NLP, đặc biệt là trong hiểu ngôn ngữ tự nhiên (NLU). Nó có khả năng thực hiện các kỳ tích ấn tượng, chẳng hạn như giải thích truyện cười, đưa ra các câu trả lời chi tiết từng bước cho các câu hỏi, và thậm chí là lập trình. Điều này một phần là do kích thước của mô

hình, nhưng cũng nhờ vào một kỹ thuật được gọi là Chain of thought prompting, được giới thiệu vài tháng trước đó bởi một nhóm các nhà nghiên cứu khác của Google.

Trong các tác vụ trả lời câu hỏi, nhắc nhở thông thường thường bao gồm một vài ví dụ về câu hỏi và câu trả lời, chẳng hạn như: “Q: Roger có 5 quả bóng tennis. Anh ấy mua thêm 2 hộp bóng tennis. Mỗi hộp có 3 quả bóng tennis. Bây giờ anh ấy có bao nhiêu quả bóng tennis? A: 11.” Lời nhắc sau đó tiếp tục với câu hỏi thực tế, chẳng hạn như “Q: John chăm sóc 10 con chó. Mỗi con chó mất 0,5 giờ mỗi ngày để đi dạo và giải quyết công việc của chúng. Anh ấy dành bao nhiêu giờ một tuần để chăm sóc chó? A:”, và công việc của mô hình là thêm câu trả lời: trong trường hợp này, “35.”

Nhưng với chain of thought prompting, các câu trả lời ví dụ bao gồm tất cả các bước lập luận dẫn đến kết luận. Ví dụ, thay vì “A: 11”, lời nhắc chứa “A: Roger bắt đầu với 5 quả bóng. 2 hộp, mỗi hộp 3 quả bóng tennis là 6 quả bóng tennis. $5 + 6 = 11$.” Điều này khuyến khích mô hình đưa ra câu trả lời chi tiết cho câu hỏi thực tế, chẳng hạn như “John chăm sóc 10 con chó. Mỗi con chó mất 0,5 giờ mỗi ngày để đi dạo và giải quyết công việc của chúng. Vì vậy, đó là $10 \times 0,5 = 5$ giờ một ngày. $5 \text{ giờ một ngày} \times 7 \text{ ngày một tuần} = 35 \text{ giờ một tuần}$. Câu trả lời là 35 giờ một tuần.” Đây là một ví dụ thực tế từ bài báo!

Mô hình không chỉ đưa ra câu trả lời đúng thường xuyên hơn nhiều so với việc sử dụng nhắc nhở thông thường — chúng ta đang khuyến khích mô hình suy nghĩ kỹ lưỡng — mà nó còn cung cấp tất cả các bước lập luận, điều này có thể hữu ích để hiểu rõ hơn lý do đằng sau câu trả lời của mô hình.

Transformers đã chiếm lĩnh NLP, nhưng chúng không dừng lại ở đó: chúng sớm mở rộng sang thị giác máy tính.

Vision Transformers

Một trong những ứng dụng đầu tiên của cơ chế chú ý ngoài NMT là tạo chú thích hình ảnh bằng cách sử dụng chú ý thị giác: một mạng nơ-ron tích chập đầu tiên xử lý hình ảnh và xuất ra một số bản đồ tính năng, sau đó một bộ giải mã RNN được trang bị cơ chế chú ý tạo ra chú thích, từng từ một.

Ở mỗi bước thời gian của bộ giải mã (tức là mỗi từ), bộ giải mã sử dụng mô hình chú ý để tập trung vào đúng phần của hình ảnh. Ví dụ, trong Hình 16-12, mô hình đã tạo chú thích “A woman is throwing a frisbee in a park”, và bạn có thể thấy phần nào của hình ảnh đầu vào mà bộ giải mã đã tập trung chú ý khi nó sắp xuất ra từ “frisbee”: rõ ràng, hầu hết sự chú ý của nó được tập trung vào frisbee.



Hình 16-12. Chú ý thị giác: một hình ảnh đầu vào (trái) và tiêu điểm của mô hình trước khi tạo ra từ “frisbee” (phải)

Khi transformer ra đời vào năm 2017 và mọi người bắt đầu thử nghiệm chúng ngoài NLP, chúng lần đầu tiên được sử dụng cùng với CNN, mà không thay thế chúng. Thay vào đó, transformer thường được sử dụng để thay thế RNN, ví dụ, trong các mô hình chú thích hình ảnh. Transformer trở nên trực quan hơn một chút trong một bài báo năm 2020 của các nhà nghiên cứu Facebook, đề xuất một kiến trúc CNN–transformer lai để phát hiện đối tượng. Một lần nữa, CNN đầu tiên xử lý các hình ảnh đầu vào và xuất ra một tập hợp các bản đồ tính năng, sau đó các bản đồ tính năng này được chuyển đổi thành chuỗi và cấp cho một transformer, xuất ra các dự đoán hộp giới hạn. Nhưng một lần nữa, hầu hết công việc thị giác vẫn được thực hiện bởi CNN.

Sau đó, vào tháng 10 năm 2020, một nhóm các nhà nghiên cứu Google đã phát hành một bài báo giới thiệu một mô hình thị giác hoàn toàn dựa trên transformer, được gọi là vision transformer (ViT). Ý tưởng này đơn giản đến ngạc nhiên: chỉ cần cắt hình ảnh thành các ô vuông nhỏ 16×16 , và coi chuỗi các ô vuông như thể nó là một chuỗi các biểu diễn từ. Để chính xác hơn, các ô vuông trước tiên được làm phẳng thành các vector $16 \times 16 \times 3 = 768$ chiều — số 3 là dành cho các kênh màu RGB — sau đó các vector này đi qua một lớp tuyến tính biến đổi chúng nhưng giữ lại số chiều của chúng. Chuỗi các vector kết quả sau đó có thể được xử lý giống như một chuỗi các embedding từ: điều này có nghĩa là thêm các embedding vị trí, và truyền kết quả đến transformer. Thế là xong! Mô hình này đã đánh bại trạng thái nghệ thuật về phân loại hình ảnh ImageNet, nhưng công bằng mà nói, các tác giả đã phải sử dụng hơn 300 triệu hình ảnh bổ sung để huấn luyện. Điều này có ý nghĩa vì transformer không có nhiều bias cảm ứng như mạng nơ-ron tích chập, vì vậy chúng cần thêm dữ liệu chỉ để học những điều mà CNN ngầm định giả định.

Chỉ hai tháng sau, một nhóm các nhà nghiên cứu Facebook đã phát hành một bài báo giới thiệu các transformer hình ảnh hiệu quả dữ liệu (DeiT). Mô hình của họ đã đạt được kết quả cạnh tranh trên ImageNet mà không yêu cầu bất kỳ dữ liệu bổ sung nào để huấn luyện. Kiến trúc của mô hình hầu như giống với ViT gốc, nhưng các tác giả đã sử dụng một kỹ thuật chưng cất để truyền kiến thức từ các mô hình CNN tiên tiến nhất sang mô hình của họ.

Sau đó, vào tháng 3 năm 2021, DeepMind đã phát hành một bài báo quan trọng giới thiệu kiến trúc Perceiver. Nó là một transformer đa phương thức, nghĩa là bạn có thể cấp cho nó văn bản, hình ảnh, âm thanh hoặc hầu như bất kỳ phương thức nào khác. Cho đến lúc đó, transformer đã bị hạn chế ở các chuỗi khá ngắn do hiệu suất và nút cổ chai RAM trong các lớp chú ý. Điều này loại trừ các phương thức như âm thanh hoặc video, và nó buộc các nhà nghiên cứu phải xử lý hình ảnh dưới dạng chuỗi các mảng con, thay vì chuỗi các pixel. Nút cổ chai là do tự chú ý (self-attention), nơi mỗi token phải chú ý đến mọi token khác: nếu chuỗi đầu vào có M token, thì lớp chú ý phải tính toán một ma trận $M \times M$, có thể rất lớn nếu M rất lớn. Perceiver giải quyết vấn đề này bằng cách dần dần cải thiện một biểu diễn tiềm ẩn khá ngắn của các đầu vào, bao gồm N token — thường chỉ vài trăm. (Từ “tiềm ẩn” có nghĩa là ẩn, hoặc bên trong.) Mô hình chỉ sử dụng các lớp chú ý chéo (cross-attention layers), cấp cho chúng biểu diễn tiềm ẩn làm truy vấn, và các đầu vào (có thể lớn) làm giá trị. Điều này chỉ yêu cầu tính toán một ma trận $M \times N$, vì vậy độ phức tạp tính toán là tuyến tính đối với M , thay vì bậc hai. Sau khi đi qua một số lớp chú ý chéo, nếu mọi thứ diễn ra tốt đẹp, biểu diễn tiềm ẩn cuối cùng sẽ nắm bắt được mọi thứ quan trọng trong các đầu vào. Các tác giả cũng gợi ý chia sẻ trọng số giữa các lớp chú ý chéo liên tiếp: nếu bạn làm điều đó, thì Perceiver thực sự trở thành một RNN. Thật vậy, các lớp chú ý chéo được chia sẻ có thể được xem là cùng một ô nhớ ở các bước thời gian khác nhau, và biểu diễn tiềm ẩn tương ứng với vector ngữ cảnh của ô. Cùng một đầu vào được cấp liên tục cho ô nhớ ở mỗi bước thời gian. Có vẻ như RNN chưa chết!

Chỉ một tháng sau, Mathilde Caron et al. đã giới thiệu DINO, một transformer hình ảnh ẩn tượng được huấn luyện hoàn toàn không nhãn, sử dụng tự giám sát, và có khả năng phân đoạn ngữ nghĩa độ chính xác cao. Mô hình được nhân đôi trong quá trình huấn luyện, với một mạng hoạt động như một giáo viên và mạng kia hoạt động như một học sinh. Gradient descent chỉ ảnh hưởng đến học sinh, trong khi trọng số của giáo viên chỉ là trung bình động theo cấp số nhân của trọng số của học sinh. Học sinh được huấn luyện để khớp với các dự đoán của giáo viên: vì chúng gần như là cùng một mô hình, điều này được gọi là tự chưng cất. Ở mỗi bước huấn luyện, các hình ảnh đầu vào được tăng cường theo những cách khác nhau cho giáo viên và học sinh, vì vậy chúng không nhìn thấy cùng một hình ảnh chính xác, nhưng các dự đoán của chúng phải khớp. Điều này buộc chúng phải đưa ra các biểu diễn cấp cao. Để ngăn chặn sự sụp đổ chế độ (mode collapse), nơi cả học sinh và giáo viên sẽ luôn xuất ra cùng một thứ, hoàn toàn bỏ qua đầu vào, DINO theo dõi trung bình động của đầu ra của giáo viên, và nó điều chỉnh các dự đoán của giáo viên để đảm bảo rằng chúng vẫn tập trung vào số 0, trung bình. DINO cũng buộc giáo viên phải có độ tin cậy cao trong các dự đoán của mình: điều này được gọi là làm sắc nét (sharpening). Cùng nhau, các kỹ thuật này duy trì sự đa dạng trong đầu ra của giáo viên.

Trong một bài báo năm 2021, các nhà nghiên cứu Google đã chỉ ra cách thay đổi kích thước ViT lên hoặc xuống, tùy thuộc vào lượng dữ liệu. Họ đã tạo ra một mô hình khổng lồ 2 tỷ tham số đạt độ chính xác top-1 trên 90,4% trên ImageNet. Ngược lại, họ cũng đã huấn luyện một mô hình giảm kích thước đạt độ chính xác top-1 trên 84,8% trên ImageNet, chỉ sử dụng 10.000 hình ảnh: đó chỉ là 10 hình ảnh cho mỗi lớp!

Và sự tiến bộ trong các transformer thị giác vẫn tiếp tục ổn định cho đến ngày nay. Ví dụ, vào tháng 3 năm 2022, một bài báo của Mitchell Wortsman et al. đã chứng minh rằng có thể huấn luyện nhiều transformer trước, sau đó tính trung bình trọng số của chúng để tạo ra một mô hình mới và cải tiến. Điều này tương tự như một ensemble (xem Chương 7), ngoại trừ cuối cùng chỉ có một mô hình, nghĩa là không có hình phạt thời gian suy luận.

Xu hướng mới nhất trong transformer bao gồm việc xây dựng các mô hình đa phương thức lớn, thường có khả năng học zero-shot hoặc few-shot. Ví dụ, bài báo CLIP năm 2021 của OpenAI đã đề xuất một mô hình transformer lớn được tiền huấn luyện để khớp chú thích với hình ảnh: tác vụ này cho phép nó học các biểu diễn hình ảnh xuất sắc, và mô hình sau đó có thể được sử dụng trực tiếp cho các tác vụ như phân loại hình ảnh bằng các lời nhắc văn bản đơn giản như “a photo of a cat”. Ngay sau đó, OpenAI đã công bố DALL·E, có khả năng tạo ra những hình ảnh tuyệt vời dựa trên các lời nhắc văn bản. DALL·E 2, tạo ra những hình ảnh chất lượng cao hơn nữa bằng cách sử dụng mô hình khuếch tán (xem Chương 17).

Vào tháng 4 năm 2022, các nhà nghiên cứu Google đã phát hành bài báo Flamingo, giới thiệu một họ mô hình được tiền huấn luyện trên nhiều tác vụ khác nhau trên nhiều phương thức, bao gồm văn bản, hình ảnh và video. Một mô hình duy nhất có thể được sử dụng trên các tác vụ rất khác nhau, chẳng hạn như trả lời câu hỏi, tạo chú thích hình ảnh, v.v. Ngay sau đó, vào tháng 5 năm 2022, DeepMind đã giới thiệu GATO, một mô hình đa phương thức có thể được sử dụng làm chính sách cho một tác nhân học tăng cường (RL sẽ được giới thiệu trong Chương 18). Cùng một transformer có thể trò chuyện với bạn, chú thích hình ảnh, chơi trò chơi Atari, điều khiển cánh tay robot (mô phỏng), v.v., tất cả chỉ với “chi” 1,2 tỷ tham số. Và cuộc phiêu lưu vẫn tiếp tục!

Như bạn có thể thấy, transformer có mặt ở khắp mọi nơi! Và tin tốt là bạn thường sẽ không phải tự triển khai transformer vì nhiều mô hình tiền huấn luyện xuất sắc đã có sẵn để tải xuống thông qua TensorFlow Hub hoặc trung tâm mô hình của Hugging Face. Bạn đã thấy cách sử dụng một mô hình từ TF Hub, vì vậy hãy kết thúc chương này bằng cách xem nhanh hệ sinh thái của Hugging Face.

Thư viện Transformers của Hugging Face

Không thể nói về transformer ngày nay mà không nhắc đến Hugging Face, một công ty AI đã xây dựng toàn bộ hệ sinh thái các công cụ mã nguồn mở để sử dụng cho NLP, thị giác, và hơn thế nữa. Thành phần trung tâm của hệ sinh thái của họ là thư viện Transformers, cho phép bạn dễ dàng tải xuống một mô hình tiền huấn luyện, bao gồm bộ mã hóa tương ứng của nó, và sau đó tinh chỉnh nó trên tập dữ liệu của riêng bạn, nếu cần. Thêm vào đó, thư viện hỗ trợ TensorFlow, PyTorch và JAX (với thư viện Flax).

Cách đơn giản nhất để sử dụng thư viện Transformers là sử dụng hàm `transformers.pipeline()`: bạn chỉ cần chỉ định tác vụ bạn muốn, chẳng hạn như phân tích cảm xúc, và nó sẽ tải xuống một mô hình tiền huấn luyện mặc định, sẵn sàng để sử dụng — thực sự không thể đơn giản hơn:

```
from transformers import pipeline

classifier = pipeline("sentiment-analysis") # nhiều tác vụ khác có sẵn
result = classifier("The actors were very convincing.")
```

Kết quả là một danh sách Python chứa một từ điển cho mỗi văn bản đầu vào:

```
>>> result
[{'label': 'POSITIVE', 'score': 0.9998071789741516}]
```

Trong ví dụ này, mô hình đã tìm thấy chính xác rằng câu là tích cực, với độ tin cậy khoảng 99,98%. Tất nhiên, bạn cũng có thể cấp một batch các câu cho mô hình:

```
>>> classifier(["I am from India.", "I am from Iraq."])
[{'label': 'POSITIVE', 'score': 0.9896161556243896},
 {'label': 'NEGATIVE', 'score': 0.9811071157455444}]
```

Thiên vị và Công bằng

Như đầu ra gợi ý, bộ phân loại cụ thể này yêu thích người Ấn Độ, nhưng lại bị thiên vị nghiêm trọng đối với người Iraq. Bạn có thể thử đoạn mã này với quốc gia hoặc thành phố của riêng bạn. Sự thiên vị không mong muốn như vậy thường phần lớn đến từ chính dữ liệu huấn luyện: trong trường hợp này, có rất nhiều câu tiêu cực liên quan đến các cuộc chiến tranh ở Iraq trong dữ liệu huấn luyện. Sự thiên vị này sau đó đã được khuếch đại trong quá trình tinh chỉnh vì mô hình buộc phải chọn giữa hai lớp: tích cực hoặc tiêu cực. Nếu bạn thêm một lớp trung tính khi tinh chỉnh, thì sự thiên vị quốc gia hầu hết sẽ biến mất. Nhưng dữ liệu huấn luyện không phải là nguồn gốc duy nhất của sự thiên vị: kiến trúc của mô hình, loại hàm mất mát hoặc chuẩn hóa được sử dụng để huấn luyện, bộ tối ưu hóa; tất cả những điều này đều có thể ảnh hưởng đến những gì mô hình học được. Ngay cả một mô hình hầu hết không thiên vị cũng có thể được sử dụng theo cách thiên vị, giống như các câu hỏi khảo sát có thể bị thiên vị.

Hiểu về sự thiên vị trong AI và giảm thiểu các tác động tiêu cực của nó vẫn là một lĩnh vực nghiên cứu tích cực, nhưng một điều chắc chắn: bạn nên dừng lại và suy nghĩ trước khi vội vàng triển khai một mô hình vào sản xuất. Tự hỏi bản thân xem mô hình có thể gây hại như thế nào, ngay cả gián tiếp. Ví dụ, nếu các dự đoán của mô hình được sử dụng để quyết định có cho ai đó vay tiền hay không, quy trình phải công bằng. Vì vậy, hãy đảm bảo rằng bạn đánh giá hiệu suất của mô hình không chỉ trung bình trên toàn bộ tập kiểm tra, mà còn trên các tập con khác nhau: ví dụ, bạn có thể thấy rằng mặc dù mô hình hoạt động rất tốt trung bình, nhưng hiệu suất của nó rất tệ đối với một số nhóm người. Bạn cũng có thể muốn chạy các thử nghiệm phản thực tế: ví dụ, bạn có thể muốn kiểm tra xem các dự đoán của mô hình có thay đổi hay không khi bạn chỉ cần thay đổi giới tính của ai đó.

Nếu mô hình hoạt động tốt trung bình, thật hấp dẫn để đẩy nó vào sản xuất và chuyển sang việc khác, đặc biệt nếu nó chỉ là một thành phần của một hệ thống lớn hơn nhiều. Nhưng nói chung, nếu bạn không khắc phục những vấn đề như vậy, sẽ không ai khác làm, và mô hình của bạn có thể gây hại nhiều hơn lợi. Giải pháp phụ thuộc vào vấn đề: nó có thể yêu cầu cân

bằng lại tập dữ liệu, tinh chỉnh trên một tập dữ liệu khác, chuyển sang một mô hình tiền huấn luyện khác, điều chỉnh kiến trúc hoặc siêu tham số của mô hình, v.v.

Hàm `pipeline()` sử dụng mô hình mặc định cho tác vụ đã cho. Ví dụ, đối với các tác vụ phân loại văn bản như phân tích cảm xúc, tại thời điểm viết bài, nó mặc định là `distilbert-base-uncased-finetuned-sst-2-english` — một mô hình DistilBERT với bộ mã hóa không phân biệt chữ hoa chữ thường, được huấn luyện trên Wikipedia tiếng Anh và một corpus các sách tiếng Anh, và được tinh chỉnh trên tác vụ Stanford Sentiment Treebank v2 (SST-2). Cũng có thể chỉ định thủ công một mô hình khác. Ví dụ, bạn có thể sử dụng một mô hình DistilBERT được tinh chỉnh trên tác vụ Multi-Genre Natural Language Inference (MultiNLI), phân loại hai câu thành ba lớp: mâu thuẫn, trung tính hoặc suy luận. Đây là cách thực hiện:

```
model_name = "huggingface/distilbert-base-uncased-finetuned-mnli"
classifier_mnli = pipeline("text-classification", model=model_name)
classifier_mnli("She loves me. [SEP] She loves me not.")
# Output: [{'label': 'contradiction', 'score': 0.9790192246437073}]
```

API pipeline rất đơn giản và tiện lợi, nhưng đôi khi bạn sẽ cần kiểm soát nhiều hơn. Đối với những trường hợp như vậy, thư viện Transformers cung cấp nhiều lớp, bao gồm tất cả các loại bộ mã hóa, mô hình, cấu hình, callbacks, và nhiều hơn nữa. Ví dụ, hãy tải cùng mô hình DistilBERT, cùng với bộ mã hóa tương ứng của nó, bằng cách sử dụng các lớp `TFAutoModelForSequenceClassification` và `AutoTokenizer`:

```
from transformers import AutoTokenizer, TFAutoModelForSequenceClassification

tokenizer = AutoTokenizer.from_pretrained(model_name)
model = TFAutoModelForSequenceClassification.from_pretrained(model_name)
```

Tiếp theo, hãy mã hóa một vài cặp câu. Trong đoạn mã này, chúng ta kích hoạt padding và chỉ định rằng chúng ta muốn các tensor TensorFlow thay vì các danh sách Python:

```
token_ids = tokenizer(["I like soccer. [SEP] We all love soccer!",
                      "Joe lived for a very long time. [SEP] Joe is old."],
                      padding=True, return_tensors="tf")
```

Đầu ra là một instance giống từ điển của lớp `BatchEncoding`, chứa các chuỗi ID token, cũng như một mặt nạ chứa các số 0 cho các token đệm:

```
>>> token_ids
{'input_ids': <tf.Tensor: shape=(2,15), dtype=int32, numpy=
array([[ 101, 1045, 2066, 4715, 1012, 102, 2057, 2035, 2293, 4715, 999,
        102,    0,    0,    0],
       [ 101, 3533, 2973, 2005, 1037, 2200, 2146, 2051, 1012, 102, 3533,
        2003, 2214, 1012, 102]]), dtype=int32)>,
 'attention_mask':
<tf.Tensor: shape=(2, 15), dtype=int32, numpy= array([[1, 1, 1, 1, 1, 1, 1,
        1, 1, 1, 1, 1, 0, 0, 0],
       [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1]]), dtype=int32)>}
```

Nếu bạn đặt `return_token_type_ids=True` khi gọi tokenizer, bạn cũng sẽ nhận được một tensor bổ sung cho biết mỗi token thuộc về câu nào. Điều này cần thiết cho một số mô hình, nhưng không phải DistilBERT.

Tiếp theo, chúng ta có thể trực tiếp truyền đối tượng `BatchEncoding` này cho mô hình; nó trả về một đối tượng `TFSequenceClassifierOutput` chứa các logits lớp được dự đoán của nó:

```
>>> outputs = model(token_ids)
>>> outputs
TFSequenceClassifierOutput(loss=None,
logits=[<tf.Tensor: [...] numpy= array([[ -2.1123817 ,  1.1786783 ,  1.4101017
],
[ -0.01478387,  1.0962474 , -0.9919954 ]], dtype=float32)>], [...])
```

Cuối cùng, chúng ta có thể áp dụng hàm kích hoạt softmax để chuyển đổi các logits này thành xác suất lớp, và sử dụng hàm `argmax()` để dự đoán lớp có xác suất cao nhất cho mỗi cặp câu đầu vào:

```
>>> Y_probas = tf.keras.activations.softmax(outputs.logits)
>>> Y_probas
<tf.Tensor:
shape=(2, 3), dtype=float32, numpy= array([[0.01619702, 0.43523544, 0.5485676
],
[0.08672056, 0.85204804, 0.06123142]], dtype=float32)>

>>> Y_pred = tf.argmax(Y_probas, axis=1)
>>> Y_pred # 0 = contradiction, 1 = entailment, 2 = neutral
<tf.Tensor:shape=(2,), dtype=int64, numpy=array([2, 1])>
```

Trong ví dụ này, mô hình phân loại đúng cặp câu đầu tiên là trung tính (việc tôi thích bóng đá không có nghĩa là mọi người khác cũng thích) và cặp thứ hai là suy luận (Joe thực sự phải khá già).

Nếu bạn muốn tinh chỉnh mô hình này trên tập dữ liệu của riêng bạn, bạn có thể huấn luyện mô hình như bình thường với Keras vì nó chỉ là một mô hình Keras thông thường với một vài phương thức bổ sung. Tuy nhiên, vì mô hình xuất ra logits thay vì xác suất, bạn phải sử dụng hàm mất mát `tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True)` thay vì hàm mất mát thông thường `"sparse_categorical_crossentropy"`. Hơn nữa, mô hình không hỗ trợ đầu vào `BatchEncoding` trong quá trình huấn luyện, vì vậy bạn phải sử dụng thuộc tính `data` của nó để lấy một từ điển thông thường thay thế:

```
sentences = [("Sky is blue", "Sky is red"), ("I love her", "She loves me")]
X_train = tokenizer(sentences, padding=True, return_tensors="tf").data
y_train = tf.constant([0, 2]) # contradiction, neutral

loss = tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True)
model.compile(loss=loss, optimizer="nadam", metrics=["accuracy"])
history = model.fit(X_train, y_train, epochs=2)
```

Hugging Face cũng đã xây dựng một thư viện Datasets mà bạn có thể sử dụng để dễ dàng tải xuống một tập dữ liệu tiêu chuẩn (như IMDb) hoặc một tập dữ liệu tùy chỉnh, và sử dụng nó để tinh chỉnh mô hình của bạn. Nó tương tự như TensorFlow Datasets, nhưng nó cũng cung cấp các công cụ để thực hiện các tác vụ tiền xử lý phổ biến một cách nhanh chóng, chẳng hạn như che mặt. Danh sách các tập dữ liệu có sẵn tại <https://huggingface.co/datasets>.

Điều này sẽ giúp bạn bắt đầu với hệ sinh thái của Hugging Face. Để tìm hiểu thêm, bạn có thể truy cập <https://huggingface.co/docs> để xem tài liệu, bao gồm nhiều sổ tay hướng dẫn, video, API đầy đủ, v.v. Tôi cũng khuyên bạn nên xem cuốn sách của O'Reilly “Natural Language Processing with Transformers: Building Language Applications with Hugging Face” của Lewis Tunstall, Leandro von Werra, và Thomas Wolf — tất cả đều từ nhóm Hugging Face.

Trong chương tiếp theo, chúng ta sẽ thảo luận về cách học các biểu diễn sâu một cách không giám sát bằng cách sử dụng autoencoder, và chúng ta sẽ sử dụng mạng đối nghịch tạo sinh để tạo ra hình ảnh và nhiều hơn nữa!

Bài tập

1. Ưu và nhược điểm của việc sử dụng RNN có trạng thái so với RNN không trạng thái là gì?
2. Tại sao người ta sử dụng RNN mã hóa-giải mã thay vì RNN tuần tự thông thường để dịch tự động?
3. Làm thế nào bạn có thể xử lý các chuỗi đầu vào có độ dài thay đổi? Còn các chuỗi đầu ra có độ dài thay đổi thì sao?
4. Beam search là gì, và tại sao bạn lại sử dụng nó? Bạn có thể sử dụng công cụ nào để triển khai nó?
5. Cơ chế chú ý là gì? Nó giúp ích như thế nào?
6. Lớp quan trọng nhất trong kiến trúc transformer là gì? Mục đích của nó là gì?
7. Khi nào bạn cần sử dụng softmax lấy mẫu?
8. Ngữ pháp Reber nhúng được Hochreiter và Schmidhuber sử dụng trong bài báo của họ về LSTM. Chúng là các ngữ pháp nhân tạo tạo ra các chuỗi như “BPBTSXXVPSEPE”. Hãy xem phần giới thiệu hay của Jenny Orr về chủ đề này, sau đó chọn một ngữ pháp Reber nhúng cụ thể (chẳng hạn như ngữ pháp được biểu diễn trên trang của Orr), sau đó huấn luyện một RNN để xác định xem một chuỗi có tuân thủ ngữ pháp đó hay không. Bạn sẽ cần viết một hàm có khả năng tạo một batch huấn luyện chứa khoảng 50% chuỗi tuân thủ ngữ pháp và 50% chuỗi không tuân thủ.
9. Huấn luyện một mô hình mã hóa-giải mã có thể chuyển đổi chuỗi ngày từ định dạng này sang định dạng khác (ví dụ: từ “April 22, 2019” sang “2019-04-22”).
10. Tham khảo ví dụ trên trang web Keras về “Tìm kiếm hình ảnh ngôn ngữ tự nhiên với Bộ mã hóa kép”. Bạn sẽ học cách xây dựng một mô hình có khả năng biểu diễn cả hình ảnh và văn bản trong cùng một không gian embedding. Điều này giúp có thể tìm kiếm hình ảnh bằng lời nhắc văn bản, giống như trong mô hình CLIP của OpenAI.

11. Sử dụng thư viện Hugging Face Transformers để tải xuống một mô hình ngôn ngữ tiền huấn luyện có khả năng tạo văn bản (ví dụ: GPT), và thử tạo văn bản kiểu Shakespeare thuyết phục hơn. Bạn sẽ cần sử dụng phương thức `generate()` của mô hình — xem tài liệu của Hugging Face để biết thêm chi tiết.

Các giải pháp cho các bài tập này có sẵn ở cuối sổ tay chương này, tại <https://homl.info/colab3>.

Tài liệu tham khảo

- Alan Turing, “Computing Machinery and Intelligence”, *Mind* 49 (1950): 433–460.
- Tất nhiên, từ chatbot xuất hiện muộn hơn nhiều. Turing gọi bài kiểm tra của mình là trò chơi bắt chước: máy A và người B trò chuyện với người thẩm vấn C qua tin nhắn văn bản; người thẩm vấn hỏi các câu hỏi để tìm ra cái nào là máy (A hay B). Máy vượt qua bài kiểm tra nếu nó có thể đánh lừa người thẩm vấn, trong khi người B phải cố gắng giúp người thẩm vấn.
- Vì các cửa sổ đầu vào chồng lên nhau, khái niệm epoch không rõ ràng trong trường hợp này: trong mỗi epoch (theo cách triển khai của Keras), mô hình sẽ thực sự nhìn thấy cùng một ký tự nhiều lần.
- Alec Radford et al., “Learning to Generate Reviews and Discovering Sentiment”, arXiv preprint arXiv:1704.01444 (2017).
- Rico Sennrich et al., “Neural Machine Translation of Rare Words with Subword Units”, *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics 1* (2016): 1715–1725.
- Taku Kudo, “Subword Regularization: Improving Neural Network Translation Models with Multiple Subword Candidates”, arXiv preprint arXiv:1804.10959 (2018).
- Taku Kudo và John Richardson, “SentencePiece: A Simple and Language Independent Subword Tokenizer and Detokenizer for Neural Text Processing”, arXiv preprint arXiv:1808.06226 (2018).
- Yonghui Wu et al., “Google’s Neural Machine Translation System: Bridging the Gap Between Human and Machine Translation”, arXiv preprint arXiv:1609.08144 (2016).
- Ragged tensors được giới thiệu trong Chương 12, và chúng được chi tiết trong Phụ lục C.
- Matthew Peters et al., “Deep Contextualized Word Representations”, *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies 1* (2018): 2227–2237.
- Jeremy Howard và Sebastian Ruder, “Universal Language Model Fine-Tuning for Text Classification”, *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics 1* (2018): 328–339.
- Daniel Cer et al., “Universal Sentence Encoder”, arXiv preprint arXiv:1803.11175 (2018).
- Ilya Sutskever et al., “Sequence to Sequence Learning with Neural Networks”, arXiv preprint (2014).

- Samy Bengio et al., “Scheduled Sampling for Sequence Prediction with Recurrent Neural Networks”, arXiv preprint arXiv:1506.03099 (2015).
- Tập dữ liệu này bao gồm các cặp câu được tạo bởi những người đóng góp của dự án Tatoeba. Khoảng 120.000 cặp câu đã được chọn bởi các tác giả của trang web <https://manythings.org/anki>. Tập dữ liệu này được phát hành theo giấy phép Creative Commons Attribution 2.0 France. Các cặp ngôn ngữ khác cũng có sẵn.
- Trong Python, nếu bạn chạy `a, *b = [1, 2, 3, 4]`, thì `a` bằng 1 và `b` bằng `[2, 3, 4]`.
- Sébastien Jean et al., “On Using Very Large Target Vocabulary for Neural Machine Translation”, Proceedings of the 53rd Annual Meeting of the Association for Computational Linguistics and the 7th International Joint Conference on Natural Language Processing of the Asian Federation of Natural Language Processing 1 (2015): 1–10.
- Dzmitry Bahdanau et al., “Neural Machine Translation by Jointly Learning to Align and Translate”, arXiv preprint arXiv:1409.0473 (2014).
- Minh-Thang Luong et al., “Effective Approaches to Attention-Based Neural Machine Translation”, Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing (2015): 1412–1421.
- Ashish Vaswani et al., “Attention Is All You Need”, Proceedings of the 31st International Conference on Neural Information Processing Systems (2017): 6000–6010.
- Vì transformer sử dụng các lớp dense phân phối thời gian, bạn có thể cho rằng nó sử dụng các lớp tích chập 1D với kích thước kernel là 1.
- Đây là hình 1 từ bài báo “Attention Is All You Need”, được tái bản với sự cho phép của các tác giả.
- Có thể sử dụng ragged tensors thay thế, nếu bạn đang sử dụng phiên bản TensorFlow mới nhất.
- Đây là phần bên phải của hình 2 từ “Attention Is All You Need”, được tái bản với sự cho phép của các tác giả.
- Điều này rất có thể sẽ thay đổi vào thời điểm bạn đọc bài này; hãy xem Keras issue #16248 để biết thêm chi tiết. Khi điều này xảy ra, sẽ không cần đặt đối số `attention_mask`, và do đó không cần tạo `encoder_pad_mask`.
- Hiện tại `Z + skip` không hỗ trợ che mặt tự động, đó là lý do tại sao chúng ta phải viết `tf.keras.layers.Add()([Z, skip])` thay thế. Một lần nữa, điều này có thể thay đổi vào thời điểm bạn đọc bài này.
- Alec Radford et al., “Improving Language Understanding by Generative Pre-Training” (2018).
- Ví dụ, câu “Jane had a lot of fun at her friend’s birthday party” suy luận “Jane enjoyed the party”, nhưng nó bị mâu thuẫn bởi “Everyone hated the party” và nó không liên quan đến “The Earth is flat”.

- Jacob Devlin et al., “BERT: Pre-Training of Deep Bidirectional Transformers for Language Understanding”, Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies 1 (2019).
- Đây là hình 1 từ bài báo, được tái bản với sự cho phép của các tác giả.
- Alec Radford et al., “Language Models Are Unsupervised Multitask Learners” (2019).
- William Fedus et al., “Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity” (2021).
- Victor Sanh et al., “DistilBERT, A Distilled Version of Bert: Smaller, Faster, Cheaper and Lighter”, arXiv preprint arXiv:1910.01108 (2019).
- Mariya Yao đã tóm tắt nhiều mô hình này trong bài đăng này: <https://homl.info/yaopost>.
- Colin Raffel et al., “Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer”, arXiv preprint arXiv:1910.10683 (2019).
- Aakanksha Chowdhery et al., “PaLM: Scaling Language Modeling with Pathways”, arXiv preprint arXiv:2204.02311 (2022).
- Jason Wei et al., “Chain of Thought Prompting Elicits Reasoning in Large Language Models”, arXiv preprint arXiv:2201.11903 (2022).
- Kelvin Xu et al., “Show, Attend and Tell: Neural Image Caption Generation with Visual Attention”, Proceedings of the 32nd International Conference on Machine Learning (2015): 2048–2057.
- Đây là một phần của hình 3 từ bài báo. Nó được tái bản với sự cho phép của các tác giả.
- Marco Tulio Ribeiro et al., “‘Why Should I Trust You?’: Explaining the Predictions of Any Classifier”, Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (2016): 1135–1144.
- Nicolas Carion et al., “End-to-End Object Detection with Transformers”, arXiv preprint arxiv:2005.12872 (2020).
- Alexey Dosovitskiy et al., “An Image Is Worth 16x16 Words: Transformers for Image Recognition at Scale”, arXiv preprint arxiv:2010.11929 (2020).
- Hugo Touvron et al., “Training Data-Efficient Image Transformers & Distillation Through Attention”, arXiv preprint arxiv:2012.12877 (2020).
- Andrew Jaegle et al., “Perceiver: General Perception with Iterative Attention”, arXiv preprint arxiv:2103.03206 (2021).
- Mathilde Caron et al., “Emerging Properties in Self-Supervised Vision Transformers”, arXiv preprint arxiv:2104.14294 (2021).
- Xiaohua Zhai et al., “Scaling Vision Transformers”, arXiv preprint arxiv:2106.04560v1 (2021).

- Mitchell Wortsman et al., “Model Soups: Averaging Weights of Multiple Fine-tuned Models Improves Accuracy Without Increasing Inference Time”, arXiv preprint arxiv:2203.05482v1 (2022).
- Alec Radford et al., “Learning Transferable Visual Models From Natural Language Supervision”, arXiv preprint arxiv:2103.00020 (2021).
- Aditya Ramesh et al., “Zero-Shot Text-to-Image Generation”, arXiv preprint arxiv:2102.12092 (2021).
- Aditya Ramesh et al., “Hierarchical Text-Conditional Image Generation with CLIP Latents”, arXiv preprint arxiv:2204.06125 (2022).
- Jean-Baptiste Alayrac et al., “Flamingo: a Visual Language Model for Few-Shot Learning”, arXiv preprint arxiv:2204.14198 (2022).
- Scott Reed et al., “A Generalist Agent”, arXiv preprint arxiv:2205.06175 (2022).